Silent Unpairing: How Mid-Training Probes Break Same-Seed Comparisons and Can Enable Selection Bias

Mingyu Du* *z5719990@ad.unsw.edu.au*
University of New South Wales

Lu Han *lhan0123@sydney.edu.au*
The University of Sydney

Zhaoqi He
King's College London

Jingyao Zhang *jzha0544@sydney.edu.au*
The University of Sydney

Junming Huang *junming.huang@sydney.edu.au*
The University of Sydney

Yun Li *yuli0068@uni.sydney.edu.au*
The University of Sydney

Zerun Niu *zerun.niu@sydney.edu.au*
The University of Sydney

Nguyen H. Tran* *nguyen.tran@sydney.edu.au*
The University of Sydney

Ken-Tye Yong* *ken.yong@sydney.edu.au*
The University of Sydney

Reviewed on OpenReview: *https://openreview.net/forum?id=Xr662CduGR*

Abstract

Same-seed comparisons, which train a method and its baseline from the same random seed, are widely used to evaluate deep-learning methods while controlling run-to-run variation. This control matters more as AI agents (LLM-based coding and research agents, AI-scientist systems and other autonomous experimentation pipelines) increasingly run such comparisons for researchers: we argue that autonomous experiments meet the conditions for the failure we study by default, putting the reproducibility of agent-reported results at risk. The failure: a mid-training probe that runs in only one of the two runs (an evaluation pass, sampling call or monitoring hook) can advance a random-number-generator (RNG) stream or data cursor that training later reads. For probes that leave model state untouched, in an otherwise identical deterministic training program, we prove a sufficient condition for bitwise-identical training and characterize the first update at which it breaks, without ruling out later reconvergence. This desynchronization alone need not bias the average method effect. Reporting the best of several runs (probe positions, configurations or checkpoints) on the affected metric, however, cannot be biased downward in expectation: a winner's-curse effect that holds even for dependent, non-Gaussian runs with unequal means. We separate this bias from genuine differences between probe positions; under an iid Gaussian model it is $c_m\sigma$, a descriptive

*Corresponding authors.

reference scale fitted on the same runs. An audit of 28 deliberately selected codebases finds 22 exposed, four guarded and two configuration-dependent. In the strongest re-execution (Meta-Weight-Net), isolating the probe restores model and meta-network state bitwise identical to a probe-free run on 5/5 seeds. Picking the best-scoring probe position yields a 4.01-point gain over the average position in one single-machine campaign; whether this carries over to an untouched test set is unresolved, as both primary validation-to-test intervals include zero. A nanoGPT prediction fixed in advance (0.003884 loss units) is consistent with 16 new seeds (0.003431), checking mean calibration in one more setting, not transfer across settings. In autonomous experimentation, an LLM agent's progress checks, piecemeal relaunches and automated hyperparameter search can meet these conditions without deliberate choice, silently breaking the same-seed comparisons it relies on and reporting gains that may not reproduce. The repair applies unchanged to agentic research pipelines: isolate each probe's RNG use, run training deterministically, and rerun each distinct code path to confirm a zero repeat difference at a stated scope. We provide a reproducible diagnosis of a deep-learning evaluation failure, not a claim that any audited published result is wrong. Code and data: `https://github.com/dumingyu2004/silent-unpairing-tmlr2026`.

1 Introduction

The first principle is that you must not fool yourself—and you are the easiest person to fool.
— Richard Feynman, *Cargo Cult Science* (1974)

The same-seed paired comparison is widely used to evaluate deep-learning methods: to attribute a small performance difference to a method rather than run-to-run noise, one launches method and baseline from an identical seed, holds everything else fixed, and reads the gap as the method's effect. The instrument looks self-verifying: same seed, same initialization, same data order, so whatever diverges must be the method. This paper studies when that intended experimental control fails. A mid-training probe that advances a stream later consumed by training can change subsequent stochastic inputs and updates. The fixed-offset account holds on the aligned prefix; later reconvergence is possible. State-pure RNG isolation and deterministic execution restore the sufficient pairing condition, with per-path repeat checks testing its implementation. The first failure is not automatically a marginal mean bias: it is loss of the common-random-number coupling and its covariance-based variance reduction. If the shifted stream preserves the arm's marginal law, more seeds still estimate the marginal method effect; whether paired variance increases depends on the change in covariance. Directly selecting the largest reported gap has nonnegative expected reporting optimism relative to the selected placement's own marginal mean (Proposition 2), without iid or Gaussian assumptions. The observed uplift above the placement average also includes systematic placement effects, which we separate analytically. Whether this translates into a positive gain on an untouched test set after validation-based selection remains unresolved: both primary E1 intervals include zero (Section 2.4). We read twenty-eight official and field-standard codebases spanning 2018–2026, from noisy-label vision methods to LLM pretraining, and found the exposure in twenty-two, among them all eleven in the neighborhood the diagnosis came from, four of which we confirmed broken by re-execution; the selection step is the machinery those same pipelines and reporting practices already run, from best-of-m placement and configuration choices to validation-guarded champion selection. The same conditions can arise without any deliberate choice when LLM-based agents plan and run such comparisons on their own (Xu et al., 2026).

Our main finding is the *pairing-divergence artifact*. A mid-training probe that consumes a stream later used by training can change batch orders or dropout masks and break exact coupling (Proposition 1). An in-house paired comparison reporting +2.97 fell to +0.16 once the probe's RNG was isolated, and the guarded endpoint motivates the conditional no-op result (Section 2.6); its historical metric zeros are not universal bitwise certificates.

Contributions. (1) We diagnose a failure of same-seed evaluation, formalize a sufficient pairing condition and the onset of divergence, verify the tested boundary inside the HuggingFace Trainer (Figure 2), audit

exposure across twenty-eight codebases (Section 2.3: eleven core repositories double-read, twelve field-standard training loops, five flagship stacks). Separately, selection on the reported quantity yields an observed +4.01 uplift over the placement average in released code. We use $c_6\bar{\sigma}$ as a conditional iid-Gaussian benchmark, with same-sample calibration on one codebase, one dataset and a single rig (Proposition 2; Section 2.4). A separate nanoGPT experiment freezes the prediction before evaluating held-out seeds (16 seeds per split), assessing mean calibration within one additional setting rather than cross-setting transferability. Its discrepancy interval includes zero. This held-out-seed check is distinct from the validation-to-test bridge above. Monitoring and full 120-epoch witnesses extend the pairing diagnosis. (2) We give a practical repair: isolate every probe, pin kernels (nondeterministic ones reproduce the failure with no RNG sharing at all), and require an exact repeat floor. On top of it we prove a bitwise no-op certificate (Corollary 1) by which a repaired selection endpoint can prove that a comparison it declined to change is unpolluted (Section 2.6). We claim no unconditional accuracy gain for that endpoint anywhere in this paper; its payout behavior is reported in the appendix. (3) A second, appendix-scoped diagnosis (Appendix C.5) explains why this method family is the one the audit keeps landing on: the canonical lr-scaled virtual step emits an exactly η-scaled meta-signal (Proposition 3), which is potentially dominated where an η-independent term of comparable or larger size competes for the same weights. It is a magnitude ratio rather than a certainty, three named escapes mitigate it, and we hold it to a lower evidentiary bar than the rest of the paper: analytical, instrumented in three systems, not audited at scale.

Common-random-number alignment and selection optimism have established theoretical foundations; recent work treats event alignment in stochastic simulation (Buffalo et al., 2026). Our contribution is their concrete connection in deep-learning evaluation: source-level exposure audits, controlled first-consumer traces, repaired comparisons against probe-free baselines, and a quantitative assessment of placement selection. The general selection inequalities clarify the estimand; the released-code diagnosis and its verification supply the empirical contribution.

The diagnosis is constructive. The repair has three requirements with different costs: isolation is often a small code change, whereas deterministic kernels and per-path repeat/state checks add runtime, storage, and sometimes platform constraints. Under verified premises, a selection endpoint that serves its raw candidate yields the conditional no-op certificate in Corollary 1. Historical tuned-regime zero metrics are reported separately from this certificate: cells without an independent floor are excluded from certificate counts, and metric equality is not upgraded to weight or full-state identity. Section 2.6 and Figure 5 distinguish the available evidence. Code, data and analysis are released at `https://github.com/dumingyu2004/silent-unpairing-tmlr2026`; Appendix D.6 describes the release's contents and the remaining limits of historical reproduction.

2 The Pairing-Divergence Artifact

2.1 Mechanism

The evaluation question is whether a same-seed comparison preserves the intended control over training randomness. Consider two runs launched from the same seed: a method arm M and a baseline arm B. Both draw from process-global RNG streams (PyTorch CPU, per-device CUDA, NumPy, Python's built-in random). Suppose M executes a probe at step t that consumes k draws from stream σ (a meta-validation draw, a shuffled evaluation loader, a model re-initialization) and B does not. If training itself consumes σ after t, its reads are index-shifted by k on the initially aligned, equal-consumption prefix, absent further differential consumption, restoration, or reseeding. Batch orders or dropout masks can change; the runs remain aligned until an actual state update differs. Later re-coincidence is not ruled out; the witnesses document divergence at checked checkpoints or endpoints, not every-update persistence. Single-bit differences are known to drive optimization to materially different final models (Summers & Dinneen, 2021); this is a point-level effect, not a rounding curiosity. Its consequences split by estimand rather than by implementation label. Without selection, the first failure is loss of the common-random-number coupling (and hence of the covariance term that can reduce paired variance); it is not, by itself, a claim of marginal mean bias. Selection on the affected quantity is different: expected optimism relative to the selected placement's own mean is nonnegative. The max-minus-placement-average uplift also includes selection among unequal placement means. In our in-house

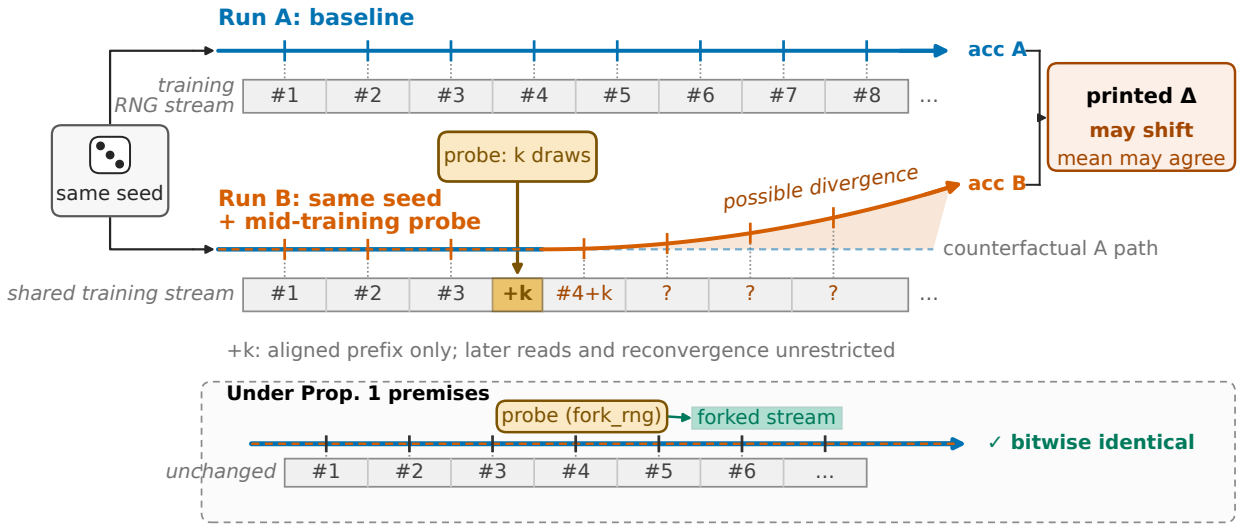


Figure 1: **An extra shared-stream consumption shifts corresponding reads on the aligned execution prefix.** The first changed training-state update breaks trajectory identity; different draws alone need not do so, and later reconvergence is possible (Proposition 1). State-pure isolation of every relevant stream restores the sufficient pairing condition.

predecessor a paired RTE gain of +2.97 (4/5 seeds) collapsed to +0.16 (2/5) under isolation. That pair predates the protocol of Section 2.5 and carries no verified repeat floor (the +2.97 arm is by construction the unisolated one), so we report it at a lower tier than the paper's floor-verified paired results. Figure 1 sketches the mechanism.

To keep these claims separate, let $U \sim P$ denote the complete training-visible random input for one seed and let $Y_M(U)$ and $Y_B(U)$ be the two endpoint metrics under the intended common input. The marginal method effect is $\tau = \mathbb{E}_P[Y_M(U)] - \mathbb{E}_P[Y_B(U)]$, whereas the paired estimator uses $Y_M(U) - Y_B(U)$. A displaced arm observes $\widetilde{\Delta} = Y_M(TU) - Y_B(U)$ for a stream-shift map T, and therefore

$$\mathbb{E}[\widetilde{\Delta}] - \tau = \mathbb{E}_P[Y_M(TU)] - \mathbb{E}_P[Y_M(U)], \qquad \mathrm{Var}(\widetilde{\Delta}) = \mathrm{Var}(Y_M(TU)) + \mathrm{Var}(Y_B(U)) - 2\,\mathrm{Cov}(Y_M(TU), Y_B(U)).$$

Thus a measure-preserving shift ($T_{\#}P = P$) leaves the marginal mean unchanged but can reduce the covariance benefit; a finite data-cursor shift need not be measure-preserving, so Proposition 1 alone does not assert marginal unbiasedness. For m placements, the pathwise selection uplift is $H = \max_j \Delta_j - m^{-1} \sum_j \Delta_j \geq 0$; calling it a bias requires the placement average to be the reference estimand. When selection uses a validation quantity and reporting uses an untouched test quantity, no one-sided conclusion follows without a transfer assumption.

2.2 An exact condition

The artifact is governed by shared stateful consumption, and the condition is testable. We state it over *streams* in a deliberately broad sense: any shared stateful sequence with a monotone cursor, covering PRNG streams and shared data iterators alike.

Proposition 1 (Pairing condition)**.** *Let X be the complete non-stream training state, including parameters, buffers, optimizer and scheduler state, modes, caches, and control state; stream values and cursors are tracked separately. Consider deterministic executions of the same training program, initially identical in both kinds of state, with aligned training operations $X_{n+1} = F_n(X_n, z_n)$, where z_n comprises the stream values read by operation n. A* probe *leaves X unchanged and can affect training only through stream cursors. Then (i) identical training reads imply bitwise-identical X trajectories; disjoint probe streams or complete fork–restore suffice. (ii) A probe advancing one stream by $k \geq 1$ shifts its subsequent training reads by k*

only along an aligned, equal-consumption prefix with no further differential advance, restoration, or reseeding. The first divergence is the first aligned operation with common input state x and $F_n(x, z) \neq F_n(x, z')$, not merely different intermediate values. Later reconvergence is possible. A probe affecting only streams never subsequently read by training is inert. (iii) Preserving reads is sufficient, not necessary: distinct read values can induce the same state update. No necessity or probability-one divergence claim is made.

Figure 2 is the paper's cleanest test of Proposition 1, because it varies only the probe inside an unmodified standard framework and reads the result at the bit level. The matched five-arm audit runs inside the HuggingFace Trainer (Wolf et al., 2020) (RoBERTa-base, RTE, 150 steps, deterministic kernels). Relative to the same reference, an independent repeat, Trainer periodic evaluation (a sequential sampler and a no-grad forward), and a CPU random-batch probe remain bitwise identical (Figure 2A). Only the CUDA random-batch probe diverges, because training's dropout later consumes that same stream. A third probe scheduled after the final update is exactly inert, realizing the boundary case of Proposition 1(ii). The reason the evaluation arm survives is *stream disjointness in this configuration*, not zero consumption: the eval loader's iterator does draw one 64-bit value from the global CPU stream at construction (`torch/utils/data/dataloader.py:706-710`), the same draw the BHN witness below turns into a divergence, but training's randomness here lives on CUDA and on a sampler with its own generator, so no training read is displaced. The CPU-probe arm measures that directly. A configuration whose training path did read the global CPU stream, a stochastic collator, or another RNG-consuming evaluation path need not be clean.

The same experiment answers how large the displacement is. Across five seeds on each of RTE and MRPC, every CUDA-probe pair diverges and every independent repeat remains exact (Figure 2B). The ten unique task–seed pairs have maximum parameter differences 5.38–6.33×10^{-4} (mean 5.92×10^{-4}). Their descriptive $|\Delta\text{acc}|$ averages 2.58 points and reaches 5.86, with inconsistent sign (4 positive, 6 negative): the artifact does not merely inflate or deflate, it scatters. At full training length the displacement persists (Figure 6: mean 1.27, maximum 2.73 points), the same order as, and often larger than, the gaps that method-comparison tables in this literature are built on.

2.3 Prevalence, and four byte-level witnesses

We audited eleven published or field-standard codebases, spanning the noisy-label and bilevel neighborhood where the diagnosis arose plus nanoGPT as a reference pretraining loop, in two static-reading waves (six classic repositories, four official, ∼4,400 stars; five recent or large-scale). Table 1 sketches the verdicts; the appendix gives file-line citations against pinned commits.

These eleven were *chosen*, not sampled. They are the neighborhood the diagnosis came from plus a reference pretraining loop, selected because we had reason to suspect them. Eleven of eleven therefore measures how uniform the idiom is inside a suspected neighborhood; it is not an estimate of any rate across deep learning, and no count here is one. The later waves are selected for reach the same way. What the audit does support is narrower: the pattern is easy to find in code people use, no repository in the core wave isolates any stream, and the exposure survives into work published through 2024.

The paper's evidence comes at three strengths and Table 1 sits in the middle. *Proven generally*: Proposition 1 holds for any run meeting its premises. *Shown dynamically*: realized divergence, confirmed by paired re-execution for four systems only; the tier column marks which. Those four are not interchangeable either, and we rank them here rather than let the count stand for them: Meta-Weight-Net and the new nanoGPT campaign compare repaired arms against a *probe-free* baseline; BHN and DivideMix verify exact same-configuration floors but contrast two configurations, and DivideMix's one line moves the mixture's initialization as well as its stream position; the historical 200-iteration nanoGPT witness had exact floors but no repaired arm. *Static, and only that*: an EXPOSED verdict says a shared-stream training consumer and a mid-training consumer of that stream are both present in the source. It says nothing about whether a published number is wrong, and nothing about any particular comparison, because desynchronizing a pair additionally needs that consumer to fire *differently* between the arms. Where it fires identically in both, the shared stream is a path this comparison does not walk. Arm-differential firing is what the four re-executed witnesses show and a static reading cannot. The harvest measurements of Section 2.4 distinguish historical single-codebase calibration from a separate nanoGPT calibration/held-out check.

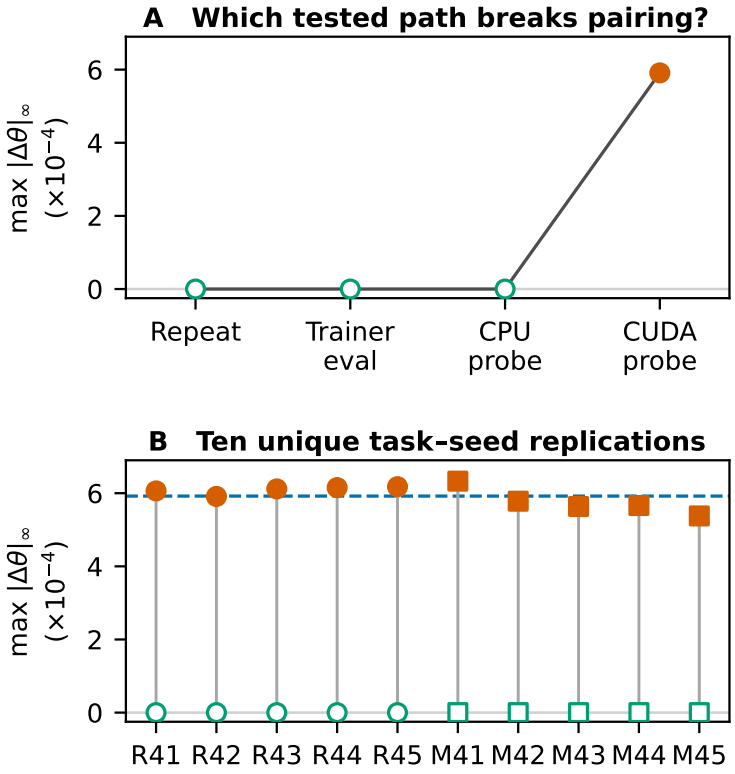


Figure 2: **Among the tested HuggingFace paths, only the probe that touches training's CUDA random stream breaks exact pairing.** (A) In one matched five-arm audit, repeat, sequential Trainer evaluation, and a CPU random-batch probe are exactly zero; the CUDA random-batch probe is nonzero. (B) Across ten unique task–seed pairs, every repeat is exact and every CUDA probe diverges. Each value is measured against its corresponding reference; no isolated-HF arm is implied.

Codebase	Exposure mechanism	Tier
L2RW official (ICML'18)	eval eats train iterator	static
L2RW PyTorch reimpl.	per-step re-init; shuffled eval	static
MW-Net official (NeurIPS'19)	per-step re-init + meta loader	**dyn.**
DivideMix official (ICLR'20)	unseeded GMM ↔ mixup	**dyn.**
Co-teaching official (NeurIPS'18)	eval-active dropout (CUDA)	static
MAML reimpl. (Finn et al., 2017)	mid-epoch shuffled test loader	static
RENT official (ICLR'24)	on-device multinomial (CUDA)	static
BHN official (ICML'23)	per-epoch loader rebuild + probe	**dyn.**
TopoFilter official (NeurIPS'20)	per-epoch probe over shuffled loader	static
CRUST official (NeurIPS'20)	train-mode gradient probe	static
nanoGPT (60.7k⋆)	eval shares training's randint	**dyn.**

Table 1: Eleven audited codebases, eleven exposures; none isolates any stream (negative grep, independently re-verified); "dyn." = divergence confirmed by re-execution.

All eleven are exposed; none constructs a dedicated generator object to isolate any stream (verified by negative grep), and seeding, where attempted, is a single global call (one reimplementation never seeds torch; CRUST (Mirzasoleiman et al., 2020) defaults to none). Every verdict was re-derived by an independent second reading against pinned commits (SHAs in the appendix): none refuted, two strengthened, since TopoFilter's (Wu et al., 2020) probe consumes a full shuffled-loader permutation per epoch, and stock DivideMix never seeds NumPy, so exact same-seed reruns of the released code are irreproducible past warm-up. Four of the eleven (Meta-Weight-Net, nanoGPT (Karpathy, 2022), DivideMix, and BHN) are additionally confirmed by dynamic paired re-execution (the L2RW reimplementation was re-executed only for signal-scale measurements, not

a paired-divergence test), and in all four, paired runs diverge exactly when the static analysis predicts. Divergence and repair are separate results, verified on different sets, and we state the split rather than average over it. Meta-Weight-Net includes a repaired arm run against a *probe-free* baseline and found SHA-256 identical to it (5/5 seeds). For BHN and DivideMix we re-executed a repaired arm and verified its same-configuration floor exactly, but the reported contrast is between two configurations rather than against a probe-free run. The historical nanoGPT witness had no repaired arm; the new campaign below adds that comparison. The DivideMix witness localizes the cause to one line: four arms of the official code (CIFAR-10, 50% symmetric noise, 12 epochs) under one byte-identical harness that seeds NumPy and pins kernels, neither of which the release does, differ only in whether the co-divide GMM draws its initialization from the shared global NumPy stream or from a pinned sklearn random state. Both repeat floors are exact; all four logs are byte-identical through the ten warm-up epochs, where the differing line never executes, then separate at the first co-divide epoch, ending 0.73 points apart on the printed final accuracy. This localizes the cause to that line and no further: pinning the mixture's random state changes its own initialization as well as where its draws come from, so the two arms differ both in the split the mixture produces and in the stream position training inherits. DivideMix is therefore a witness that the line matters, not a clean separation of cursor displacement from re-initialization; the state-pure probes of Proposition 1 are the other three. The BHN witness isolates a still smaller consumption: the single 64-bit seed its mid-training test loader draws when its iterator is constructed (the loader is unshuffled and eval-mode; that seed is its *only* shared-stream draw). One native generator argument to that loader removes exactly that draw, method semantics untouched. In four arms (official ResNet-34, CIFAR-10, 12 epochs; both repeat floors weight-exact), per-epoch weight hashes are identical through epoch 1 and diverge exactly at epoch 2, the first epoch whose shuffle order the probe's stream position can reach, ending 1.93 points apart on the printed test accuracy: a single 64-bit draw, in official ICML-2023 code. The same witness holds at the *full shipped* 200-*epoch protocol*: the isolated pair bitwise-identical through all 200 epochs, the configurations ending 82.68 vs. 83.03. The gap anneals to 0.35 points but the mechanism is intact at convergence. The fourth witness leaves supervised vision entirely: four nanoGPT arms at 200 iterations (`shakespeare_char`, fp32) differ only in whether the loss-estimation pass draws its batches from the same `randint` stream training uses, both same-configuration repeat floors are exactly zero, and the evaluation-versus-no-evaluation contrast reaches $\max|\Delta W| = 3.3 \times 10^{-2}$ (floors in Table 11).

Three structural observations follow. First, within this neighborhood the pattern is structural, not incidental: the bilevel implementations draw meta-batches from a shuffled loader on the global generator and rebuild the network every step, so the probe fires *every training step*; the artifact is a property of the method as implemented, not of an evaluation harness. Second, the mechanisms are diverse enough to defeat spot-checking: a data-pointer variant needing no RNG at all (official L2RW's evaluation consumes batches from *training's own iterator*, and its pipeline re-seeds from entropy, so its same-seed-list comparisons were never seed-controlled), an eval-active-dropout variant (Co-teaching), and a CUDA-escalation variant (RENT). Third, the pattern persists unchanged into ICML 2023, ICLR 2024, and the 60.7k-star reference pretraining loop.

The reach extends past the noisy-label neighborhood, and divides along the isolation boundary of Section 2.5. A second wave reads twelve field-standard training loops at file–line resolution (Table 5). Ten are exposed: the reference vision recipe (timm (Wightman, 2019), whose periodic validation desynchronizes the next epoch's shuffle, reproduced under torch 2.11), a widely copied diffusion template (Wang, 2020), single-file RL, LLM finetuning, and five recent selection methods (Sel-CL, MLC, ProMix, UNICON, DISC), and InfoBatch: its NumPy pruning reseed does not protect main-process torch augmentation from the unshuffled evaluation loader's base-seed draw. Two guard the audited RNG channel (a fixed-noise DCGAN buffer and deterministic Stable-Baselines3 evaluation with a separate, privately seeded environment); this does not certify state purity, since DCGAN visualization updates BatchNorm buffers. A third wave probes five flagship 2023–2026 stacks, where the boundary is a design choice rather than an era: SDXL's in-loop logging is exposed by default, TRL and a diffusers script sit one flag from either side, OLMo-core and MMSelfSup guard the audited RNG paths under their stated configuration conditions. Neither wave enters the eleven-of-eleven count.

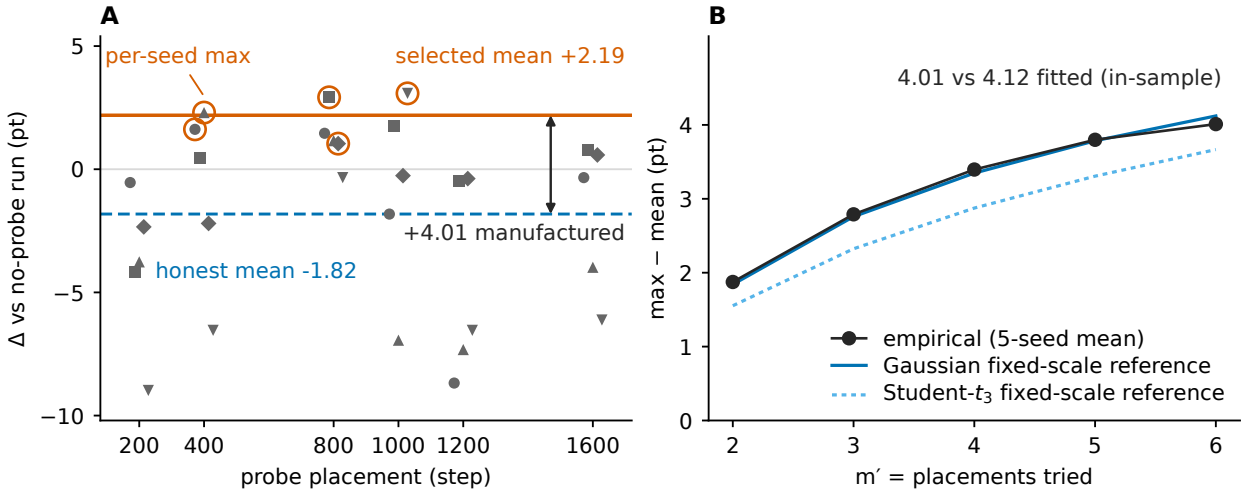


Figure 3: **Selection turns bidirectional divergence into a one-sided report.** Across five seeds and six probe placements, the honest mean is −1.82 points, whereas selecting each seed's best placement reports +2.19, manufacturing +4.01. The growth over the number of tried placements tracks the finite-m Gaussian benchmark in this grid rather than its asymptotic form (Proposition 2). This historical grid comes from one codebase on one dataset, with $c_m\sigma$ serving as a conditional, same-sample benchmark whose predictive scope is assessed separately.

2.4 The printed gap moves, and selection harvests it

Prevalence shows preconditions; a moved gap and a harvested one show consequence. The claim is tiered like the evidence: the static audit establishes the *precondition*, the byte-level witnesses the *desynchronization*, the harvest the *uplift in the selected reported quantity*. We claim no audited paper's published result is wrong, only that the instrument can be silently defeated, and that selection on the reported quantity can produce a one-sided reported uplift. In Meta-Weight-Net's official codebase, with model, meta-network, and RNG topology untouched and the repeat floor weight-exact (model and meta-network SHA-identical across same-seed reruns, five seeds), we insert one semantically inert random-permutation draw during training and recompute the method-versus-baseline gap the paper's tables would print. The gap moves by 1.78 points on seed 1 and 1.16 points on seed 2, in opposite directions. We make no claim about MW-Net's true ordering against its baseline (our protocol is shortened); the claim is that in a NeurIPS-published method's official code, a single RNG draw moves the printed comparison by more than a point above an exact floor.

To quantify the reporting consequence of placement selection, we use an idealized scaling benchmark: under the iid-Gaussian null, selection over placements has expected uplift $c_m\sigma$. Its empirical calibration is setting-specific. The historical measurements use one codebase (Meta-Weight-Net's official release) and dataset (CIFAR-10). The separate nanoGPT check below estimates its own scale from calibration seeds before held-out evaluation, assessing within-setting prediction rather than transfer across architectures, datasets, or tasks. Across 5 seeds × 6 placements on the same rig (shortened protocol: Meta-Weight-Net official code on a 10,000-image CIFAR-10 subset at 40% symmetric noise, 20 epochs of 100 steps, giving mean within-seed sample SD $\bar{\sigma} = 3.25$ points, computed across placements with denominator $m - 1$ (`ddof=1`); Figure 3), the probe *hurts* on average (honest mean −1.82) yet per-seed best-placement selection reports +2.19, an observed +4.01 uplift over the placement average. The decomposition below separates reporting optimism from systematic placement effects; these five seeds do not identify their population shares.

Proposition 2 (Harvest size)**.** *For integrable paired gaps $D = (D_1, \ldots, D_m)$, set $\mu_j = \mathbb{E}D_j$ and let J be a fixed tie-broken maximizer of D_j. With arbitrary means and dependence, selected-placement optimism satisfies*

$$\mathcal{O} := \mathbb{E}[D_J - \mu_J] \;\geq\; \max_j \mu_j - \mathbb{E}\mu_J \;\geq\; 0.$$

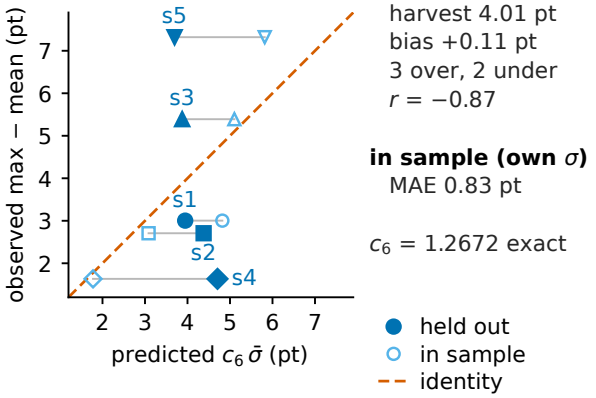


Figure 4a: **The preregistered leave-one-seed-out predictive check.** Estimating $\bar{\sigma}$ from four seeds and predicting the fifth's max-minus-mean gives a mean absolute error of 2.17 points (54% of the 4.01 being predicted), and the two series *anti*-correlate ($r = -0.87$). This observed anticorrelation is not forced by leave-one-out construction alone. Mean signed error is +0.11; substituting each seed's own fitted σ reduces MAE to 0.83 but reuses the outcomes and is not predictive validation. Neither the Gaussian shape nor transport of its scale is established by this comparison.

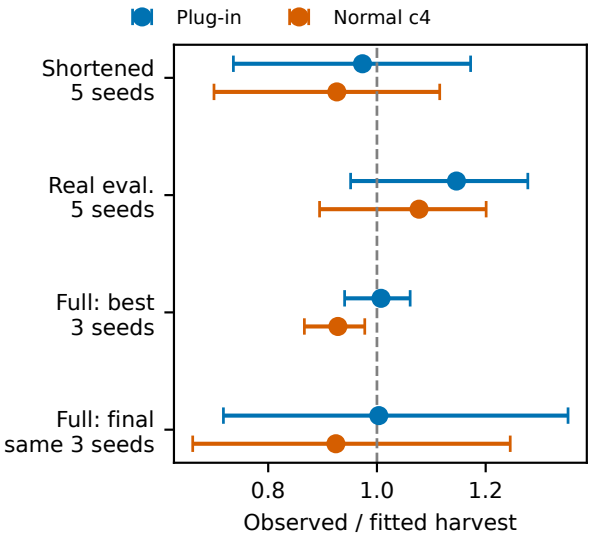


Figure 4b: **Same-sample harvest-to-scale ratios with joint whole-seed percentile** 95% **intervals, exhaustively enumerating the n^n ordered resamples and refitting the scale in each.** Both plug-in and conditional Gaussian c_4 corrections are shown. The corrected full-protocol best-checkpoint interval $[0.8665, 0.9776]$ excludes one, but has only 27 resamples from three seeds; it is not reliable evidence against the model. Best and final use the same three seeds, not independent replications.

The reported uplift $H(D) = \max_j D_j - m^{-1}\sum_j D_j$ instead obeys $\mathbb{E}H(D) = \mathbb{E}[\mu_J - \bar{\mu}] + \mathcal{O}$; the first term need not be positive. Common additive components cancel from H. In the equal-mean Gaussian special case $D_j = a + C + \sigma Z_j$, with $\sigma \geq 0$, integrable common C and i.i.d. standard normals Z_j, $\mathbb{E}H = \mathcal{O} = c_m\sigma$, where $c_m = \mathbb{E}\max_j Z_j$ ($c_6 = 1.27$). These are classical selection and order-statistic identities, not a new universal Gaussian law.

The descriptive comparison is 4.0107 points of observed uplift against $c_6\bar{\sigma} = 4.1203$, using the same placement outcomes to estimate $\bar{\sigma}$, the mean within-seed sample SD. Under the equal-mean iid-normal model, dividing each SD by its normal expectation factor $c_4(m)$ gives 4.3301. Exact quadrature yields uncorrected ratios 0.9734, 1.1463, 1.0075 across the shortened, realistic-evaluation and full-best campaigns; the corrected ratios are 0.9262, 1.0775, 0.9282. Figure 4b jointly recomputes numerator and denominator over every ordered whole-seed bootstrap sample. The corrected full-best interval is $[0.8665, 0.9776]$: it excludes one, but 27 resamples of three seeds are a discrete sensitivity analysis, not reliable model rejection or calibrated population coverage.

The placement effect previously reported at $p = 0.049$ relies on exchanging fixed training positions and is conditional on that exchangeability null. Treating observed column means as fixed offsets changed a legacy simulation ratio from 1.00 to 1.04; this is a fitted-offset sensitivity, not proof that the iid formula tolerates offsets. Appendix C now gives a distribution-free offset bound and a variance decomposition showing exactly how offsets and contrast covariance enter the fitted scale.

We additionally simulate the complete fit-and-select procedure, re-estimating all SDs in each of 100,000 Gaussian and variance-matched t_3 datasets. For the 5×6 design, the corrected-ratio reference intervals are $[0.8087, 1.1979]$ and $[0.7251, 1.3014]$, respectively. Their overlap shows why comparing expected-maximum constants with a fixed fitted SD cannot establish the residual distribution. The earlier growth curve is retained as a same-grid descriptive comparison, not distributional or predictive validation. The harvest

is consistent with an extreme-value benchmark for divergence noise, but Proposition 1(ii) alone implies neither a fresh-seed law nor Gaussian or independent placement residuals. Three limits bound how far the agreement may be read. $c_m\sigma$ prices what a *reporting practice* harvests on average over seeds rather than what any one run will show. σ is heteroscedastic across seeds (1.4–4.6) and does not transport between them, so a preregistered leave-one-seed-out test misses a held-out seed's harvest by 2.17 points on average (Figures 4a and 4b; the verification split is Figure 5). And Proposition 2 prices selection performed *on the affected quantity*: where a validation quantity selects and an untouched test quantity is reported, the direction and magnitude of transfer remain unestablished by the present evidence. The preregistered bridge gives a selected test delta of $+0.1245$. Its original preregistered seed-level t interval is $[-0.0406, 0.2896]$; the separate reviewer-requested 100,000-draw seed-cluster bootstrap gives $[-0.0180, 0.2830]$, also including zero. The within-seed centered correlation is 0.4386 (Table 12); centering removes all seed means and does not identify the baseline's causal contribution. We therefore claim a guaranteed nonnegative expected optimism only for selection on the reported quantity itself. Positive validation–test Pearson correlation alone does not guarantee positive test-selection optimism (Appendix C).

The mechanism also appears with an unmodified shuffled monitoring loader (MW-Net, 45,000 training images, 12 epochs): its two seed draws produce $+0.45$ points of max-minus-mean uplift across five seeds and five placements. The two directly checked isolated model/meta-network pairs match the never-probed baseline. At the full 120-epoch protocol, all twelve shared placement–seed pairs diverge; best-of-four uplift is $+0.16$ points, and the three checked repaired pairs match baseline. These are scoped endpoint comparisons, not all-placement trajectory verification; best and final are endpoints of the same three-seed campaign. Appendix D.7 preserves the protocols, effect sizes and verification details; Figure 5 records the coverage.

Independent-setting, frozen nanoGPT prediction check. A separate 2,000-update campaign uses five prespecified probe placements $(200, 500, 900, 1300, 1700)$, 16 calibration seeds and 16 disjoint held-out seeds. For each seed, $D_j = L_{\text{baseline}} - L_{\text{shared},j}$ uses the same fixed validation panel, and $H = \max_j D_j - \bar{D}$. Calibration alone fixes $c_5\overline{s}/c_4(5) = 0.00388406$ loss units before held-out runs; the observed held-out mean is 0.00343060. Independently bootstrapping whole seed vectors in each split (20,000 draws) gives approximate 95% intervals $[0.00327501, 0.00449714]$ for the prediction and $[0.00279334, 0.00407564]$ for the observed mean. Observed minus predicted is -0.00045346 (-11.67%), with interval $[-0.00134285, 0.00043370]$. Inclusion of zero establishes neither equivalence nor the conditional Gaussian model. This is a new setting's held-out mean-prediction check, not validation-to-test transfer or a per-run forecast. All 160 seed–placement shared paths diverge and all 160 isolated paths match their probe-free baseline at the checked state resolution; two repeats per path are verification, not additional independent observations. Appendix D.5 reports the prespecified uncorrected sensitivity and exploratory selected-score, placement and influence checks.

2.5 Kernel nondeterminism, and the repair protocol

Closing every shared-stream leak is still not sufficient for pairing. With identical training mathematics and no RNG sharing, nondeterministic GPU kernels alone bifurcate same-seed cross-arm trajectories, a documented layer (Zhuang et al., 2022) for which we claim no novelty, and one repeat checks help distinguish from deterministic stream displacement: a nonzero same-configuration floor detects nonrepeatability, whereas a deterministic stream shift can leave repeats exact while moving a cross-arm pair. A finite zero floor does not rule out all nondeterminism. The layer is material: in an under-tuned CIFAR-100 regime (constant lr, no augmentation, TF32), same-seed raw candidates across arms differed by up to 11.7 points; in our NLP suite, trajectories bitwise-identical through half an epoch later bifurcated by 13.7 points on CoLA. Seeds alone therefore never establish exact pairing; repeat checks must accompany the isolation and state-purity premises. The repair has three requirements with different costs, and they are checkable in the order given. Probe isolation is often a small code change; deterministic kernels, per-path reruns, state capture, and storage can carry substantial runtime or platform costs.

1. *Isolate every probe.* Each mid-training consumer of randomness runs inside PyTorch's fork-rng context manager (PyTorch fork rng) or uses dedicated generator objects, leaving *all* training streams untouched. All of them means all: fork-rng covers only torch's CPU/CUDA generators, so NumPy, Python's built-in random, and library-internal draws need dedicated or pinned states (DivideMix's

repair pins sklearn's); a shared *data cursor* needs its own iterator (official L2RW consumes no RNG at all); and the probe must write no training-visible state, state purity in the sense of Proposition 1. Figures 2 and 11 show the boundary this draws.

2. *Pin kernels.* Deterministic kernels for any run used in a paired claim; nondeterministic ones break pairing with no RNG sharing at all.
3. *Verify a zero repeat floor on every materially distinct path.* Rerun each arm and configuration whose kernel, control flow, precision, stochastic consumer, or probe differs, under the identical seed and configuration. Require the difference to be exactly zero at the resolution of the intended claim before interpreting a cross-arm delta (per-campaign floors: Table 11). A metric floor supports only a metric claim; weight or full-state claims require the corresponding hashes. The floor turns "we set seeds" into a measurement on the tested path, not a universal proof of determinism.

In released code the first requirement is often a small wrapper, and the three systems where we re-executed a repaired arm show the shape of it. Meta-Weight-Net's per-step probe needs the probe body wrapped in `with torch.random.fork_rng(devices=[device]):` (`devices=[]` on CPU), after which the isolated arm is SHA-256 identical to the never-probed baseline on 5/5 seeds. BHN's mid-training test loader draws one 64-bit seed at iterator construction, removed by passing `generator=torch.Generator().manual_seed(0)` to that `DataLoader`, method semantics untouched. DivideMix's co-divide GMM draws from global NumPy, pinned by `GaussianMixture(..., random_state=0)`. Each exposure in the field-wide wave maps to one of those three shapes, a `fork_rng` wrapper for an in-loop evaluation or sampling call, a dedicated `generator=` for a probe's loader, or a pinned state for a library that reads a global stream. Apart from the new nanoGPT campaign above, we have not re-executed repaired arms in that wave; for those remaining entries the mapping is a prescription, not a result.

Table 11 records the actual repeat evidence and its resolution, including absent floors. Historical NLP zero cells and the pre-protocol in-house comparison are excluded from certificate counts. A floor from one tested path is not silently transferred to a distinct arm. The isolation clause is verified on the witness codebase itself: fork-rng isolation returns model and meta-network to SHA-256 identity with the never-probed baseline, on five seeds at the shortened protocol and three at the full (probes verified to fire).

2.6 The repair's certificate: a certified no-op

The repair gives a conditional no-op result, illustrated by a guarded weight-averaging endpoint that may serve its raw candidate. Candidate maintenance and evaluation must satisfy Proposition 1; repeat floors test the implementation but do not replace its premises.

Corollary 1 (Certified no-op)**.** *Under the deterministic pairing protocol, state-pure and stream-isolated candidate maintenance gives champion = raw ⇒ served parameters and buffers are bitwise identical to the paired baseline endpoint, so every deterministic reported metric delta is exactly zero. The reverse implication requires the additional checked premise that every non-raw candidate differs from raw in at least one parameter bit; EMA or frozen candidates may collide exactly and this premise is not inferred merely from training having moved.*

The certificate requires the full conjunction of premises, not merely a logged raw champion and a zero metric. Historical tuned-regime observations are retained in Table 10 as descriptive results, with missing floors excluded from certificate counts. Direct weight comparisons, metric equality and full-state verification remain distinct evidence categories (Figure 5). Detailed averaging, monitor and meta-signal analyses are in the appendix; the main contribution is the pairing diagnosis and the operational checks that make its repair testable.

3 Related Work

Common-random-number alignment is established in stochastic simulation (Glasserman & Yao, 1992; Stout & Goldie, 2008; Pearce et al., 2022). Recent work quantifies variance reductions from alignment (Klein et al., 2024) and repairs intervention-induced stream shifts with counter-based RNG (Buffalo et al., 2026).

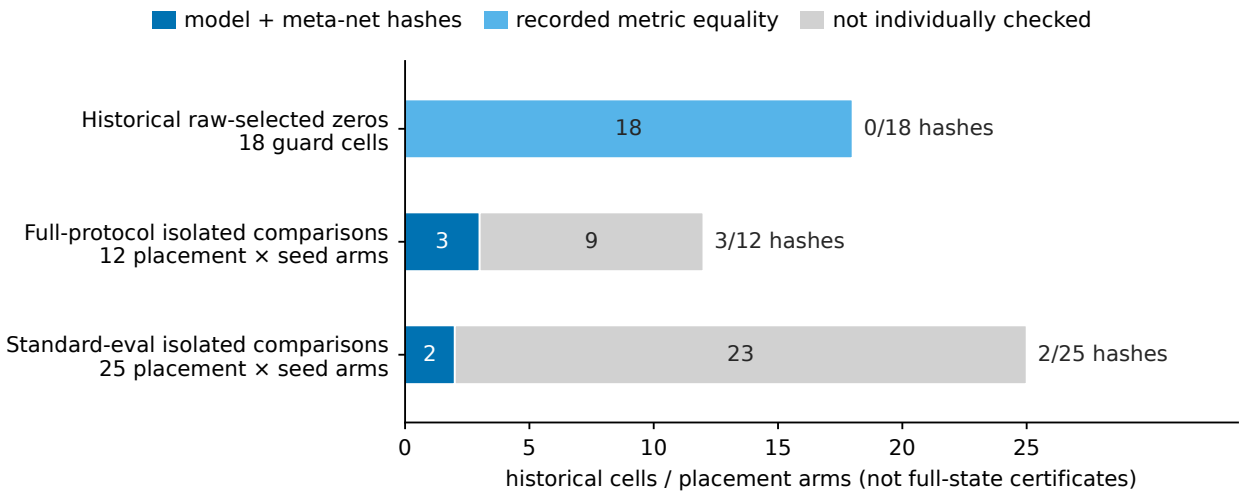


Figure 5: **Verification resolution, separated from certificate eligibility.** The full-protocol and realistic-evaluation cure counts are reproduced from archived model and meta-network hash pairs (3/12 and 2/25). Historical guard metrics and text-reported checks do not establish a full-state certificate; cells without independent floors are excluded from certificate counts. Unmeasured repaired arms remain unverified, not inferred successes.

Isolation tools already exist in JAX and PyTorch (JAX PRNG; PyTorch fork rng). We do not claim a new general principle of randomness or model selection: the contribution is a deep-learning evaluation diagnosis, released-code causal witnesses, and a connection to selection on the displaced quantity.

Deep-learning studies document sensitivity to perturbations and the need to account for variability across seeds (Summers & Dinneen, 2021; Dodge et al., 2020; Bouthillier et al., 2021). Our question complements single-run reproducibility: can an arm-specific operation invalidate the intended same-seed comparison? Deterministic-execution tools (Chen et al., 2022; Heumos et al., 2023; Shkolnikov, 2026) help check the repair, but determinism alone does not establish cross-arm pairing. Desynchronization need not alter the marginal mean effect; selecting favorable outcomes on the reported quantity invokes classical selection optimism (Cawley & Talbot, 2010).

Appendix G develops these connections, including the reweighting methods motivating the audit and the weight-averaging literature relevant to our conditional no-op application.

4 Discussion

Evaluation practice. Same-seed comparisons should check the three practices in Section 2.5: isolate probes, pin kernels, and verify each materially distinct execution path at the resolution claimed. Stream sharing alone does not imply divergence: the tested Trainer evaluation is configuration-specifically stream-disjoint from training (Table 6). Existing isolation primitives help, but their use must be checked. Deterministic execution and repeat/state audits incur runtime and storage costs; some platforms lack deterministic implementations.

Interpretation and scope. Desynchronization, variance changes, and selection optimism are distinct. Observed max-minus-mean uplift can contain both systematic placement effects and optimism; the Gaussian formula applies only under its stated model. Omitting selection removes this source of directional optimism, not trajectory displacement. The no-op corollary is an application of the pairing condition, not an accuracy claim about selection or averaging.

Our evidence is single-process and academic-scale, not distributed training, ImageNet, or production-LLM validation. The nanoGPT experiment checks a frozen mean prediction on held-out seeds in one setting, not universal Gaussian scaling, per-run prediction, or untouched-test transfer. Static exposure remains distinct

from dynamic evidence, and historical score-level equality does not establish state-level identity. Appendix G records the detailed scope and practical limitations.

5 Conclusion

Matching initial seeds does not ensure aligned training randomness. Arm-specific probes can displace later training draws; our pairing condition guarantees preservation and localizes divergence onset on the aligned prefix without ruling out later re-coincidence. Released-code witnesses make this evaluation failure concrete. Desynchronization alone need not bias the expected effect: selection on the affected reported quantity creates a separate optimism mechanism, while positive untouched-test transfer remains unestablished. The general selection decomposition accommodates placement effects; Gaussian scaling is a conditional benchmark, with a separate held-out mean check in nanoGPT. The repair is checkable through probe isolation, deterministic execution, and per-path repeat checks at the claimed resolution.

Broader Impact Statement

This paper studies how experiments are measured rather than proposing a model, a dataset, or a deployed system, so its effect is on research practice. Two consequences deserve naming. First, the audit names repositories and commits: we report structural preconditions and, where we re-executed, realized divergence, but claim no published result is wrong. These are careful implementations, written when nothing in the tooling or the review process asked for probe-RNG isolation, and the same reading returned the same verdict on our own predecessor first. Second, a procedure that inflates a reported gap is a recipe as well as a diagnosis. Disclosure is safer: the mechanism is already live in widely used released code, so it operates whether or not practitioners intend it. Isolation is often a small code change, but the full repair also requires deterministic execution and per-path repeat checks, with runtime, storage and platform costs.

References

Niccolò Ajroldi, Antonio Orvieto, and Jonas Geiping. When, where and why to average weights? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 925–941. PMLR, 2025.

Gül Sena Altıntaş, Devin Kwok, Colin Raffel, and David Rolnick. The butterfly effect: Neural network training trajectories are highly sensitive to initial conditions. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 1314–1342. PMLR, 2025.

HeeSun Bae, Seungjae Shin, Byeonghu Na, and Il-Chul Moon. Dirichlet-based per-sample weighting by transition matrix for noisy label learning. In *The Twelfth International Conference on Learning Representations*, 2024.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, pp. 747–769, 2021.

Vince Buffalo, Carl A. B. Pearson, and Daniel Klein. Realizing common random numbers: Event-keyed hashing for causally valid stochastic models, 2026. arXiv:2603.11084.

Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010.

Boyuan Chen, Mingzhi Wen, Yong Shi, Dayi Lin, Gopi Krishnan Rajbahadur, and Zhen Ming Jiang. Towards training reproducible deep learning models. In *Proceedings of the 44th International Conference on Software Engineering*, pp. 2202–2214. ACM, 2022.

Caglar Demir, Arnab Sharma, and Axel-Cyrille Ngonga Ngomo. Adaptive stochastic weight averaging, 2024. arXiv:2406.19092.

Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah A. Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping, 2020. arXiv:2002.06305.

William Falcon and The PyTorch Lightning team. Pytorch lightning. `https://github.com/Lightning-AI/pytorch-lightning`, 2019. Software.

Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1126–1135. PMLR, 2017.

Paul Glasserman and David D. Yao. Some guidelines and guarantees for common random numbers. *Management Science*, 38(6):884–908, 1992.

Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pp. 3207–3214, 2018.

Lukas Heumos, Philipp Ehmele, Luis Kuhn Cuellar, Kevin Menden, Edmund Miller, Steffen Lemke, Gisela Gabernet, and Sven Nahnsen. mlf-core: a framework for deterministic machine learning. *Bioinformatics*, 39(4):btad164, 2023.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *Proceedings of the Thirty-Fourth Conference on Uncertainty in Artificial Intelligence*, pp. 876–885. AUAI Press, 2018.

JAX PRNG. JEP 263: JAX PRNG design (splittable keys). `https://docs.jax.dev/en/latest/jep/263-prng.html`, 2018. Software-design document.

Yilin Ji, Daniel Kaestner, Oliver Wirth, and Christian Wressnegger. Randomness is the root of all evil: More reliable evaluation of deep active learning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 3943–3952, January 2023.

Keller Jordan. On the variance of neural network training with respect to test sets and distributions. In *The Twelfth International Conference on Learning Representations*, 2024.

Jean Kaddour. Stop wasting my time! saving days of imagenet and bert training with latest weight averaging. In *NeurIPS 2022 Workshop on Has It Trained Yet? (HITY)*, 2022.

Andrej Karpathy. nanogpt. `https://github.com/karpathy/nanoGPT`, 2022. Software.

Daniel J. Klein, Romesh G. Abeysuriya, Robyn M. Stuart, and Cliff C. Kerr. Noise-free comparison of stochastic agent-based simulations using common random numbers, 2024. arXiv:2409.02086.

Junnan Li, Richard Socher, and Steven C. H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020.

Tao Li, Zhehao Huang, Qinghua Tao, Yingwen Wu, and Xiaolin Huang. Trainable weight averaging: Efficient training by optimizing historical solutions. In *The Eleventh International Conference on Learning Representations*, 2023.

Andrew Ly and Pulin Gong. Riddled basin geometry sets fundamental limits to predictability and reproducibility in deep learning, 2025. arXiv:2510.05606.

Gábor Melis. Two-tailed averaging: Anytime, adaptive, once-in-a-while optimal weight averaging for better generalization, 2022. arXiv:2209.12581.

Baharan Mirzasoleiman, Kaidi Cao, and Jure Leskovec. Coresets for robust training of deep neural networks against noisy labels. In *Advances in Neural Information Processing Systems*, volume 33, pp. 11465–11477. Curran Associates, Inc., 2020.

Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pp. 48–53, 2019.

Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318):1–63, 2024.

Michael Arthur Leopold Pearce, Matthias Poloczek, and Juergen Branke. Bayesian optimization allowing for common random numbers. *Operations Research*, 70(6):3457–3472, 2022.

PyTorch fork rng. torch.random.fork_rng. `https://pytorch.org/docs/stable/random.html`, 2026. PyTorch documentation (software).

PyTorch Lightning contributors. `TrainingEpochLoop._should_check_val_fx` discrepancy between continued run <> restore from ckpt (issue #14579). `https://github.com/Lightning-AI/pytorch-lightning/issues/14579`, 2022. The thread reports that interleaving validation and training differently across runs leaves the two consuming different random numbers when both draw from one generator.

Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. Learning to reweight examples for robust deep learning. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 4334–4343. PMLR, 2018.

Sunny Sanyal, Atula Tejaswi Neerkaje, Jean Kaddour, Abhishek Kumar, and Sujay Sanghavi. Early weight averaging meets high learning rates for llm pre-training. In *First Conference on Language Modeling (COLM)*, 2024.

Yakov Shkolnikov. Bit-identical medical deep learning via structured orthogonal initialization, 2026. arXiv:2603.28040.

Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Natasha K. Stout and Sue J. Goldie. Keeping the noise down: Common random numbers for disease simulation modeling. *Health Care Management Science*, 11(4):399–406, 2008.

Cecilia Summers and Michael J. Dinneen. Nondeterminism and instability in neural network optimization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9913–9922. PMLR, 2021.

The MathWorks, Inc. rlevaluator: Options for evaluating reinforcement learning agents during training (reinforcement learning toolbox documentation). `https://www.mathworks.com/help/reinforcement-learning/ref/rl.evaluation.rlevaulator.html`, 2024. Accessed 2026-07-17.

Peng Wang, Shengchao Hu, Zerui Tao, Guoxia Wang, Dianhai Yu, Li Shen, Quan Zheng, and Dacheng Tao. Sewa: Selective weight average via probabilistic masking, 2025. arXiv:2502.10119.

Phil Wang. denoising-diffusion-pytorch, 2020. GitHub: lucidrains/denoising-diffusion-pytorch.

Ross Wightman. Pytorch image models. `https://github.com/huggingface/pytorch-image-models`, 2019. Software.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 23965–23998. PMLR, 2022.

Pengxiang Wu, Songzhu Zheng, Mayank Goswami, Dimitris Metaxas, and Chao Chen. A topological filter for learning with label noise. In *Advances in Neural Information Processing Systems*, volume 33, pp. 21382–21393. Curran Associates, Inc., 2020.

Yuanyuan Xu, Wenjie Zhang, Yin Chen, Xuemin Lin, and Ying Zhang. Self-evolving agents as dynamic graph transformation: A survey and new perspective, 2026. Preprints.org, doi:10.20944/preprints202606.1695.v1.

Chenglin Yu, Xinsong Ma, and Weiwei Liu. Delving into noisy label detection with clean data. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 40290–40305. PMLR, 2023.

Yiming Zhang, Chester Holtz, Gal Mishne, and Alex Cloninger. Revisiting meta-learning with noisy labels: Reweighting dynamics and theoretical guarantees, 2025. arXiv:2510.12209.

Donglin Zhuang, Xingyao Zhang, Shuaiwen Song, and Sara Hooker. Randomness in neural network training: Characterizing the impact of tooling. In *Proceedings of Machine Learning and Systems*, volume 4, pp. 316–336, 2022.

The appendix is organized as follows. Appendix A is the static-audit register: per-repository mechanisms cited by file and line against pinned commits, verdicts, and the corrections an independent second reading produced. Appendix B verifies at source level the main text's two framework claims, the tested HuggingFace Trainer stream disjointness and Proposition 1's PyTorch stream premises. Appendix C proves the three propositions, the certified-no-op corollary and the margin-calibration lemma, and states the paper's second, appendix-scoped diagnosis in Appendix C.5. Appendix D gives the pairing protocol, the per-campaign repeat-floor registry, the traced MW-Net divergence witness, and the preregistered validation-to-test bridge; Appendices E and F are the environment manifest and a note on the monitor figures. Every SHA, citation, and quotation was read against the pinned version stated, and main-text statements are restated before their proofs.

A Static Audit: Pinned Commits, Mechanisms, and the Double-Read Register

Tone, stated once and applying to every named repository. These verdicts convict a practice, not people. The audited releases are careful implementations of influential methods, and we know of no venue checklist or framework default from their era that asked for probe-RNG isolation. The same reading applied to our own predecessor system returned the same verdict first.

Method. Two static-reading waves over shallow clones pinned by commit SHA: six classic method repositories (four official releases; ∼4,400 stars combined; Table 2) and five recent or large-scale targets (Table 3). A third wave (Table 4) reads five flagship 2023–2026 stacks as a *boundary probe* rather than a prevalence claim: three-way verdicts (exposed / guarded / ambiguous with the deciding flag stated), a single reading whose load-bearing citations were independently re-verified against the pinned clones, entering neither the eleven-repository count nor the corrections register. Static reading establishes only that the consumption pattern exists, a claim about code rather than about any published number; realized divergence comes from

the re-execution witnesses, whose floors are registered in Appendix D, and the remaining six verdicts are labeled static in the main text. An independent second reading against the same pinned commits refuted 0/11, strengthened two (TopoFilter, DivideMix), and produced seven secondary corrections, an eighth surfacing during witness construction (register below).

The shared negative result. None of the eleven codebases constructs a dedicated `torch.Generator` for any `DataLoader`, wraps any probe in `fork_rng`, or save-restores RNG state around any mid-training consumer; each relies on a single global seed call (or none). For the recent five this was re-verified in the second reading with a widened negative grep over all `.py` files; the pattern

```
torch.Generator|generator=|fork_rng|get_rng_state|set_rng_state
```

returns zero hits, and all 14 seeding sites in those five repositories are global `manual_seed`-family calls.

Corrections register (seven corrections from the double-read, an eighth from witness construction; zero verdicts refuted).

1. *L2RW official*: provenance inversion, the executed `DataSet` coming from TensorFlow r1.10 with the vendored copy dead code (mechanism byte-identical); an `nval` confusion clarified. Lacking a graph-level seed it re-seeds from entropy, so the repository's same-seed-list comparisons (`mnist_train.py:417-422`) were never seed-controlled, strengthening the "paired in intent, desynchronized in fact" reading.
2. *L2RW PyTorch*: periodic evaluation is cadence-symmetric across arms; the arm-differential consumer is the per-step re-instantiation, as first stated.
3. *Meta-Weight-Net*: on `torch` 0.4.0 the loader-creation `randperm` shift does not hold (lazy `BatchSampler`); the first differential consumers are the per-step rebuild (`:200`) and meta-batch augmentation (`:215`). The flip-noise path also ignores `-seed`; step-0 conclusion and verdict stand.
4. *Co-teaching*: selection uses `np.argsort`, not `torch.sort`; both RNG-free, so nothing changes. CPU-stream non-contamination re-verified under `torch` 0.3.0.
5. *RENT*: `TV.py:41`'s `randint` draws CPU-side then moves to device; the CUDA draws are `PDN.py:191` and `BLTM.py:96`. Table 3's headline mechanism is unaffected.
6. *TopoFilter*: exposure understated, the probe itself iterating a shuffled trainloader every epoch (`:269`) for a full permutation plus augmentation draws.
7. *CRUST*: facility-location weights are unnormalized counts (`:215`) and `-seed` defaults to `None`, so default runs are entirely unseeded.
8. *BHN*: the lines first cited (`:407-408`, `:430`) are the Clothing1M path; the default CIFAR-10 path exposes the same mechanism at `:756`, `:781`, `:810`, later confirmed dynamically (Appendix D).

One further correction is typographical and uncounted: nanoGPT's `estimate_loss` spans `train.py:215-228`, a one-line offset. Net: 0/11 verdicts refuted, 2 exposures strengthened, 8 mechanism-level corrections (7 double-read, 1 from witness construction), none touching a verdict.

Scope of the verdicts. EXPOSED is a claim that the preconditions of Proposition 1(ii) are present in the code: a global-stream (or shared-cursor) training consumer plus a mid-training consumer of the same stream. For Table 4, GUARDED means the probe exists but is isolated by design, and AMBIGUOUS means exposure is decided by a stated flag or argument. None of these is, by itself, a claim that any specific published number is wrong, and we make no such claim for any audited repository. Nor is any of it a rate. Every repository here was picked deliberately, the first wave because it is the neighborhood the diagnosis came from and the later waves for reach across eras and task families, so the counts describe what we looked at and support no inference to a population of repositories we did not. The realized-divergence and harvest claims of the main text rest on the re-execution witnesses and our own campaigns; Appendix D records which paths have repeat floors and at what resolution.

Codebase (venue)	Commit	Exposure mechanism (file:line)	Verdict	Double-read note
L2RW official, TF1 (ICML 2018)	`0b616c9`	Periodic eval pulls 50 batches from *training's own iterator* (`mnist_train.py:396-397` → `:270-272`), advancing the shared epoch cursor; `DataSet` construction re-seeds global NumPy (`mnist/mnist.py:151-153`), whose shuffles the loaders share (`:214-216`, `:230-232`); per-step meta-validation draw (`mnist_train.py:372-373`)	EXPOSED (data cursor + NumPy global)	Corrected (register 1); verdict unchanged, and the paired-in-intent reading strengthened
L2RW PyTorch reimpl.	`182aad0`	Per-step meta-network re-instantiation (notebook cell 17) draws dozens of initialization samples from the global CPU stream; test loader shuffled by an unconditional override (`data_loader.py:105-111`); fresh `iter()`, one `randperm`, every training step	EXPOSED (CPU stream)	Corrected (register 2); verdict unchanged
Meta-Weight-Net official (NeurIPS 2019)	`3f1800f`	Meta-loader iterator created before the train iterator and re-created mid-epoch (`MW-Net.py:196-197`, `:214-218`); full model re-instantiation every step (`:200-201`); meta-batch fetches advance the shared Python-`random` augmentation stream	EXPOSED (CPU stream, per step)	Corrected (register 3); step-0 desynchronization conclusion stands
DivideMix official (ICLR 2020)	`d9d3058`	`GaussianMixture` fitted with no `random_state` (`Train_cifar.py:186-187`) draws its initialization from global NumPy, interleaved with training's `np.random.beta` mixup draws (`:87`); NumPy never seeded; per-epoch loader rebuilds with GMM-dependent split sizes (`:264-272`)	EXPOSED (NumPy + structural)	Confirmed; zero-hit repo-wide grep for `np.random.seed`; same-seed reruns of the stock code are irreproducible past warm-up. Subsequently confirmed dynamically (four-arm witness, Appendix D): pinning the GMM's `random_state` (one line) moves the final weights, with both repeat floors exact and warm-up logs byte-identical
Co-teaching official (NeurIPS 2018)	`7c7fbe2`	Functional dropout without a `training` argument (`model.py:45`, `:54`) stays active under `model.eval()` (the API default is `training=True`); the per-epoch test evaluation (`main.py:295`) therefore consumes the CUDA stream shared with training-time dropout	EXPOSED (CUDA stream)	Corrected (register 4); verdict unchanged
MAML-PyTorch reimpl.	`98a00d4`	A shuffled test `DataLoader` is constructed mid-training every 500 steps (`miniimagenet_train.py:78-86`), drawing `randperm` from the global CPU generator shared with the training loader (`:67`)	EXPOSED (CPU stream)	Confirmed; Python-`random` unseeded repo-wide (episode composition itself irreproducible)

Table 2: Classic wave: six method codebases, six exposures (four official releases; ∼4,400 stars combined). Static reading of pinned commits; every verdict re-derived by an independent second reading, with corrections listed in the register.

Codebase (venue)	Commit	Exposure mechanism (file:line)	Verdict	Double-read note
nanoGPT (60.7k stars)	`3adf61e`	The single data path `get_batch` draws `torch.randint` with no `generator=` (`train.py:123`); `estimate_loss` reuses it for exactly 400 draws per evaluation (2 splits × 200 `eval_iters`; 4,800 sampled positions at batch 12); all three call sites (`:222`, `:250`, `:303`) share the stream; evaluation fires at iteration 0 as well	EXPOSED (CPU; canonical)	Confirmed; call-site enumeration verified exhaustive; one line range corrected (see register)
RENT official (ICLR 2024)	`d40f3c4`	On-device `torch.multinomial` every training batch (`method_utils.py:122-123`) advances the global CUDA stream by a data-dependent amount; train loader `shuffle=True` with no generator	EXPOSED (CUDA escalation)	Corrected (register 5); consumption chain additionally verified by execution
BHN official (ICML 2023)	`2b09d3a`	The default CIFAR-10 path's train loader is built once (`BHN.py:756`) and re-iterated every epoch (`:781`), drawing per-sample augmentation and the loader's own shuffle seed from the global CPU generator; every epoch ends with a mid-training test (`:810`), whose `test_loader` construction (`:209-210`) draws one further global-stream value at iterator creation with no `generator=`	EXPOSED (CPU stream)	Corrected (register 8): the lines first cited (`:407-408`, `:430`) belong to `PvalueTrainer Clothing1M`, the Clothing1M path, not to the audited default CIFAR-10 path `PvalueTrainer`. Verdict unchanged, and confirmed dynamically (Appendix D)
TopoFilter official (NeurIPS 2020)	`9835d8c`	Per-epoch feature/connected-component probe interleaved with training (`train.py:227-263`); training loaders `shuffle=True` with no generator (`:103`, `:350`)	EXPOSED (CPU stream)	Confirmed and strengthened (register 6): the probe itself iterates a shuffled trainloader each epoch (`:269`)
CRUST official (NeurIPS 2020)	`a430069`	Per-epoch train-mode gradient-estimation pass and coreset re-selection mutate the dataset the shuffled loader sees (`robust_cifar_train.py:201-237`; loader `:176`); per-epoch `validate()`	EXPOSED (CPU + structural)	Corrected (register 7); default `-seed` is `None`, so default runs are entirely unseeded

Table 3: Recent wave: five recent or large-scale codebases, five exposures. The pattern persists unchanged into ICML 2023, ICLR 2024, and the reference LLM pretraining loop; the same negative grep applies (zero isolation hits across all five).

A.1 Second wave: field-standard training loops

The main text's claim that the exposure reaches past the noisy-label neighborhood rests on a second audit of twelve field-standard codebases spanning diffusion, vision recipes, reinforcement learning, LLM finetuning, and recent sample-selection methods. We separate it from the double-read core by verification tier: this wave is a single structured reading at pinned commits, of which the four headline verdicts (`timm`, `lucidrains/ddpm`, Sel-CL, MLC) were independently re-verified at file–line by the authors, and `timm`'s data-order channel was reproduced empirically under `torch` 2.11 (inserting an eval-loader iteration between epochs shifts the next epoch's train shuffle). Confidence is stated per row; these repositories do not enter the eleven-of-eleven count.

B Framework Source-Level Verification

B.1 HuggingFace Trainer: tested stream disjointness (`transformers` 5.12.1)

The main text reports that the Trainer's built-in periodic evaluation is bitwise clean in the five-arm experiment. That measurement has a source-level explanation, verified against the installed `transformers` 5.12.1 (quotes condensed to the load-bearing expression).

Stack (paradigm)	Commit	Mechanism / isolation (file:line)	Verdict	Boundary note
HF TRL (RLHF post-training)	`b6ff4a7`	Generation-based evaluation routes through the same `generate(do_sample=True)` as training rollouts, on the global torch stream (`grpo_trainer.py:1614`; config built `:959-972` with no generator); post-generation `torch.randperm` unisolated (`trainer/utils.py:831`). The dataloader *is* isolated: a dedicated `Generator` in `RepeatSampler` (`utils.py:712-719`)	ambiguous	exposed on the default backend; `use_vllm=True` (default `False`, `grpo_config.py:574`) moves generation to an isolated engine
AI2 OLMo-core (LLM pretraining)	`fa6c501`	The in-loop evaluator forces `model.eval()` before every eval forward (`train_module.py:497`, `:649-654`), gating off the only training-mode global draws (MoE jitter `router.py:331`, `:336`; dropout defaults off); batch order is counter-keyed from a dedicated PCG64 generator (`data_loader.py:667-688`)	guarded	no audited configuration flips it. The $k = 0$ counts the training-mode global draws the evaluator gates off, not every value an evaluation path could take; what carries the verdict is the dedicated generator, which leaves training's batch order disjoint from the global stream whatever an eval loader draws from it
Stability generative-models / SDXL (diffusion)	`e8cd657`	`ImageLogger.on_train_batch_end` (`main.py:457-459`) samples images via `torch.randn` (`diffusion.py:246`) plus per-step `randn_like` on the global stream that training's noise and timesteps consume (`loss.py:72`; `sigma_sampling.py:12`, `:29`); negative greps: `fork_rng`, `isolate_rng`, `torch.Generator`, `generator=`, RNG state save/restore all zero hits	exposed	on by default (`batch_frequency` 1000, installed as a default callback)
OpenMMLab MMSelfSup (vision SSL)	`db861f3`	The DeepCluster extract loader is `shuffle=False` (config `:55`), so its *sampler* draws nothing (`torch.arange`, mmengine `sampler.py:75-76`); its iterator still takes the one `_base_seed` value on fresh iterator construction (`dataloader.py:706-710`), which is harmless here only because training shuffle uses a private `Generator` (`deepcluster_sampler.py:47-48`; mmengine `sampler.py:72-74`); augmentation randomness is confined to per-worker-seeded subprocesses (mmengine `dataset/utils.py:36-39`); persistent workers reuse existing iterators	guarded	the one config-gated global-NumPy draw (`set_uniform_indices`, `:88`) self-reseeds per epoch and lives in main-process NumPy, which training augmentation never reads. The hook also updates labels and initializes its classifier; this RNG verdict does not assert state purity
HF diffusers (training examples, diffusion)	`d1b988b`	`log_validation`'s pipeline receives `generator = None if args.seed is None else torch.Generator(...)` (`train_text_to_image.py:161-164`, consumed `:174`); `randn_tensor` draws the global stream iff the generator is `None` (`torch_utils.py:232`), the stream training consumes (`:972`, `:982`)	ambiguous	the boundary is exactly `-seed`: passed → guarded; omitted → every validation image shifts subsequent noise and timesteps

Table 4: Third wave (boundary probe): five flagship 2023–2026 stacks across RLHF post-training, LLM pretraining, diffusion, and self-supervised vision. Unlike the two prevalence waves this wave makes no all-exposed claim and its verdicts are three-way; it was a single reading whose every load-bearing citation was independently re-verified against the pinned clones. The split lands on the isolation primitives of the repair protocol, and these five do not enter the eleven-repository count.

Codebase (kind)	Commit	Shared-stream mechanism: training consumer → mid-training consumer (file:line)	Verdict	Conf.
Sel-CL (CVPR'22)	`98cc41e`	`WeightedRandomSampler` on global torch, no generator (`train_Sel-CL.py:193`) → per-epoch `pair_selection` draws `torch.rand(len(trainloader.dataset), args.low_dim)`, 50,000 × 128 on CIFAR (`utils/utils_noise.py:359`)	EXPOSED	high
MLC (AAAI'21)	`5747d0b`	silver (train) loader `shuffle=True`, no generator (`CIFAR/data_helper_cifar.py:76`) → per-epoch validation loader `shuffle=True` (`:78`) draws a full global permutation	EXPOSED	high
ProMix (IJCAI'23)	`40d8378`	mixup `torch.randperm` + train `shuffle=True` (`Train_promix.py:208`; `dataloader_cifarn.py:276`) → selection `eval_train` iterates a multi-worker loader (`Train_promix.py:427`), one worker base-seed draw/epoch	EXPOSED	med
UNICON (CVPR'22)	`1b5a03f`	mixup `randperm` + train `shuffle=True` (`Train_cifar.py:148`) → `Calculate_JSD` eval loader worker base-seed (`dataloader_cifar.py:340`)	EXPOSED	med
DISC (CVPR'23)	`a21e910`	mixup `randperm` + train `shuffle=True` (`losses/loss_mixup.py:27`, called from `DISC.py:308`) → per-epoch `model.evaluate` loader worker base-seed (`main.py:204`)	EXPOSED	med
lucidrains DDPM (diffusion, ∼9k⋆)	`d93647a`	forward-diffusion `torch.randint/randn_like` (`denoising_diffusion_pytorch.py:838`) → mid-training `ema.sample()` draws `torch.randn` (`:686`, `:709`) on the same global stream, default `every 1000`	EXPOSED	high
timm (vision recipe, ∼33k⋆)	`ce7c976`	train loader `shuffle`, no generator (`data/loader.py:348`) → `validate()` (`train.py:1392`) advances the base seed on first iterator construction; persistent reuse is not a fresh draw every epoch. Nondistributed shuffle path; *empirically reproduced*	EXPOSED	high
CleanRL (RL, ∼6k⋆)	`fe8d8a0`	online ϵ-greedy `random.random()` (`qdagger_dqn_atari_impalacnn.py:362`) → periodic offline `evaluate` reuses global `random` (`dqn_eval.py:28`)	EXPOSED	high
litgpt (LLM finetune, ∼12k⋆)	`2685705`	LoRA `Dropout` on global torch (`lora.py:82`; default `lora_dropout=0.05`, `finetune/lora.py:58`) → periodic `generate_example` temperature sampling `torch.multinomial` (`generate/base.py:36`); dataloader *is* guarded, generation is not	EXPOSED	high
InfoBatch (ICLR'24, ∼346⋆)	`8bbb012`	unshuffled test loader, no generator (`research/cifar_example.py:76-77`), consumes a CPU base seed; next-epoch random flip/crop (`:51-54`) runs with zero workers (`:70-71`). NumPy pruning reseed does not isolate torch augmentation	EXPOSED	high
PyTorch DCGAN example (∼23k⋆)	`acc295d`	training `torch.randn` (`dcgan/main.py:240`) → periodic visualization reuses a precomputed `fixed_noise` buffer (`:211`), drawing zero new noise. RNG-only guard: train-mode `netG(fixed_noise)` (`:268`) updates BatchNorm buffers	GUARDED	high
Stable-Baselines3 (∼10k⋆)	`8908708`	rollout action sampling on global torch → `EvalCallback` uses a separate `eval_env` + `deterministic=True` default (`callbacks.py:379`); assumes private environment randomness and deterministic policy internals. Stochastic actions or custom global-RNG components are outside this guard	GUARDED	med

Table 5: Second-wave audit of twelve selected training loops: ten EXPOSED, two guarded on the stated RNG channel. Re-reading the pinned InfoBatch CIFAR path changes its earlier guarded verdict to exposed: counter-reseeding NumPy does not isolate torch CPU augmentation from a loader base seed. Across all 28 paths the revised counts are 22 exposed, four guarded and two configuration-dependent. Guarded does not imply state purity or universal framework immunity. Exact source-chain conditions and before/after verdicts are recorded in the revision audit ledger.

Claim	Location	Source text (condensed)
Evaluation sampling is sequential	`trainer.py:1067-1068`	`_get_eval_sampler` returns `SequentialSampler(eval_dataset)`
A sequential sampler draws nothing	`torch/utils/data/sampler.py:109-110`	`__iter__` is `iter(range(len(...)))`: no RNG call
But the iterator itself draws one value, shuffled or not	`torch/utils/data/dataloader.py:706-710`	`_BaseDataLoaderIter.__init__` sets `_base_seed` by calling `.random_(generator=loader.generator)` on a scalar tensor; with the default `generator=None` that reads the global CPU stream, on every iterator construction
The evaluation forward consumes no stream	`trainer.py:2697`; `:2958`	`model.eval()` before the loop; `torch.no_grad()` in `prediction_step`; the loop body (`:2636-2830`) contains no `rand*`, `bernoulli`, or `multinomial` call
Worker seeding is train-side only	`trainer.py:996-999`	`worker_init_fn = seed_worker` attached only when `is_training`
The seeded train sampler lives in `accelerate`	`accelerate/data_loader.py:1194`, `:88`, `:99-101`	`SeedableRandomSampler` (default `use_seedable_sampler=True`) re-seeds each epoch with `self.epoch + self.initial_seed`
With the seedable sampler off, the asymmetry is stronger	`torch/utils/data/sampler.py:162-163`	`RandomSampler` with no generator draws one seed per epoch from the global CPU stream

Table 6: Why the tested Trainer evaluation path leaves the pair bitwise intact, cited to source. The reason is *stream disjointness in this configuration*, not zero consumption: the eval iterator does draw one 64-bit value from the global CPU stream (row 3, the same draw the BHN witness of Section 2 is built on), but training's randomness here is CUDA-side dropout plus an `accelerate` sampler with its own generator, so nothing training reads is displaced. The five-arm experiment's CPU-probe arm measures exactly that: CPU draws are inert in this configuration. A configuration whose training path did read the global CPU stream would not be covered. Versions: `transformers` 5.12.1 over `torch` 2.11.0; GitHub `main` (fetched 2026-07-07) is behaviorally identical at every cited point.

	Premise	Source citation
P1	Factory calls without `generator=` dispatch to the process-global default generator	`ATen/core/Generator.h:159-168` (default-generator dispatch)
P2	`fork_rng` saves the CPU state and the states of listed devices on entry and restores them on exit	`torch/random.py:209-217`
P3	Draws through an explicit `torch.Generator` leave the global defaults untouched	dispatch site as P1; exercised by the two explicit-generator assertions of the check script
P4	The CPU default and each CUDA device's default are distinct generators	`torch/cuda/random.py:41` (per-device default generators)

Table 7: The stream premises of Proposition 1, cited to `torch` 2.11.0 source. No premise remains unverified.

Two precision notes on the main text's phrasing. The seeded training-side object is `accelerate`'s `SeedableRandomSampler`, not a Trainer-owned generator, so the attribution belongs to the `accelerate` layer. And the measurement covers only the tested model, collator and evaluation machinery (the sequential sampler plus the no-grad forward), with a user-supplied stochastic collator the documented exception (`DataCollatorForLanguageModeling` draws masks at evaluation time); our classification collators draw nothing.

B.2 PyTorch stream premises (`torch` 2.11.0)

Proposition 1 treats PyTorch's global generators as streams in the sense of Appendix C.1. The four premises this requires are verified in source and exercised empirically.

`scripts/rng_premise_check.py` packages the premises as 13 boolean assertions: three for global dispatch (P1: `rand`, `randint`, and `randperm` each advance the global CPU state), three for fork-restore (P2: CPU state restored, CUDA state restored, and the next draw after `fork_rng` identical to the no-fork draw), two for explicit-generator isolation (P3: global CPU and CUDA states unchanged), and five for device separation (P4:

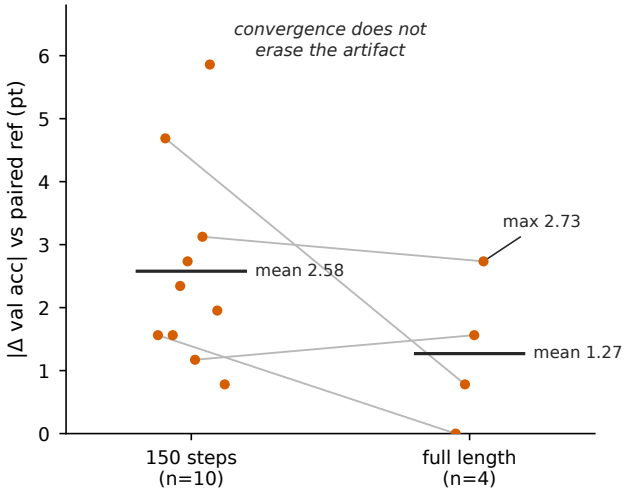


Figure 6: **Persistence of the probe displacement at convergence.** Each dot is one run's $|\Delta$ val acc$|$ between the CUDA-probe arm and its paired reference: ten 150-step pairs (mean 2.58 points) versus four full-length runs (mean 1.27, maximum 2.73); gray lines connect matched (task, seed) pairs. This is the main text's claim that convergence shrinks but does not erase the artifact.

distinct generator objects; a CUDA draw advances CUDA state and leaves CPU state unchanged; a CPU draw advances CPU state and leaves CUDA state unchanged). All 13 pass on the manifest stack (Appendix E), exercising both the CPU and the CUDA device. A semantic detail confirmed at source: `fork_rng` always forks the CPU stream, and its `devices` argument governs accelerators only, so a probe isolated by `fork_rng` is complete in both CPU-only and CUDA execution.

C Proofs

C.1 Setting: streams, cursors, probes

Notation follows the main text: θ parameters, η learning rate, Δ signed paired difference, m probe realizations, n probe-split size, s seed.

Deterministic execution (K). All paired experiments run under deterministic kernels (Appendix D): every operation's output is a fixed function of its inputs, so a run is a deterministic function of its initial state and of the values it reads from streams.

Streams as cursored sequences. A *stream* is a shared stateful sequence with a monotone cursor: an ordered value sequence $V = (v_1, v_2, \dots)$ and a cursor c, where consuming k items returns $v_c, \dots, v_{c+k-1}$ and advances $c \mapsto c + k$; outside explicit restoration or reseeding, only consumption moves c. Two families instantiate this in training. (a) *PRNG streams*: deterministic state machines $(v_i, s_{i+1}) = f(s_i)$ with s_1 fixed by the seed, so V is a function of the seed: the process-global PyTorch CPU generator, each per-device CUDA generator, the NumPy global state, and Python's `random` (Table 7). (b) *Data cursors*: shared iterators whose cursor is an epoch pointer over a data order. The breadth is load-bearing: the official L2RW evaluation desynchronizes its comparison by advancing a data cursor, consuming no PRNG draw at all (Table 2, row 1).

Training-visible state; probes. Write X for the complete *non-stream* training state: parameters, buffers (including normalization statistics), gradients, optimizer and scheduler state, modes, caches, and control state, including anything else read by a later training operation. Stream sequences and cursors are separate state, not part of X. Their read values are explicit inputs z_n to an aligned deterministic update $X_{n+1} = F_n(X_n, z_n)$. An operation block interleaved with training is a *probe* if (P-a) it writes no non-stream training state X and (P-b) interacts with training only by possibly advancing stream cursors. An evaluation pass that updates

BatchNorm running statistics, or any block that steps the optimizer, is not a probe; Corollary 1 therefore carries state purity of the framework's internal machinery as an explicit rather than tacit premise.

C.2 Proposition 1: the pairing condition

Proposition (Pairing condition)**.** *Let X be the complete non-stream training state, with stream sequences and cursors tracked separately. Consider deterministic executions of the same training program, initially identical in both kinds of state, with aligned updates $X_{n+1} = F_n(X_n, z_n)$. A probe leaves X unchanged and can affect training only through stream cursors. Then (i) identical training reads imply bitwise-identical X trajectories; disjoint probe streams or complete fork–restore suffice. (ii) A probe advancing one stream by $k \geq 1$ shifts its subsequent training reads by k only along an aligned, equal-consumption prefix with no further differential advance, restoration, or reseeding. The first divergence is the first aligned operation with common input state x and $F_n(x, z) \neq F_n(x, z')$, not merely different intermediate values. Later reconvergence is possible. A probe affecting only streams never subsequently read by training is inert. (iii) Preserving reads is sufficient, not necessary: distinct read values can induce the same state update. No necessity or probability-one divergence claim is made.*

Proof. **(i)** Induct over aligned training operations. The initial X states are identical. Probes leave X unchanged; equal X_n and equal reads z_n give equal $F_n(X_n, z_n)$ by determinism, including the control state needed to align the next operation. Disjoint streams or restoration of every touched training stream preserve those reads. Thus X trajectories and deterministic evaluations using identical evaluation inputs coincide. This does not claim equality of un-restored, training-irrelevant stream cursors.

(ii) Immediately after the probe, the affected cursor difference is k. Equal subsequent consumption counts preserve that difference until a further differential advance, restoration, or reseeding; hence reads at i_j are compared with reads at $i_j + k$ on this prefix. Starting from equal X, equality persists exactly while the realized update outputs agree. Consequently its first failure occurs at the first actual inequality $F_n(x, z) \neq F_n(x, z')$. A changed mask or batch alone does not establish this inequality. After divergence, neither aligned control flow nor equal consumption counts are guaranteed. Non-injective later updates can map different states to the same state, so reconvergence is not excluded. If no affected stream is subsequently read by training, all training reads are preserved and (i) applies.

(iii) Sufficiency is (i). Necessity fails even for a nonconstant update: $F(x, z) = x + z^2$ gives identical updates at $z = 1$ and $z = -1$. Cancelled contributions, clipping, and rounding supply further many-to-one maps. Dependence on a draw is not injectivity in that draw. No distributional claim about these equalities is required for the repair's sufficient condition. □

Lemma 1 (Coupling versus marginal estimands)**.** *Let $U \sim P$ be the random input to two deterministic endpoint maps Y_M, Y_B, with the expectations below finite (and second moments finite for the variance identity), and let T be the stream-shift map in the probed arm. Define $\tau = \mathbb{E}_P[Y_M(U)] - \mathbb{E}_P[Y_B(U)]$ and $\widetilde{\Delta} = Y_M(TU) - Y_B(U)$. Then, without any invariance assumption,*

$$\mathbb{E}[\widetilde{\Delta}] - \tau = \mathbb{E}_P[Y_M(TU)] - \mathbb{E}_P[Y_M(U)], \quad \mathrm{Var}(\widetilde{\Delta}) = \mathrm{Var}(Y_M(TU)) + \mathrm{Var}(Y_B(U)) - 2\,\mathrm{Cov}(Y_M(TU), Y_B(U)).$$

If $T_{\#}P = P$, the first quantity is zero, but the covariance term need not equal the intended paired covariance. For a data cursor, $T_{\#}P = P$ is not automatic and no marginal-unbiasedness conclusion follows from stream algebra alone. For a history-dependent intervention after a history $\mathcal{H}_t = h$, the corresponding sufficient condition is preservation of the conditional *future-input law: $(T_h)_{\#}P_h = P_h$ almost surely, where $P_h = \mathcal{L}(U_{>t} \mid \mathcal{H}_t = h)$. This must hold for the complete joint future input used by the continuation.*

Proof. Expand the expectation and variance of $Y_M(TU) - Y_B(U)$ and subtract the definition of τ. Measure preservation gives $\mathbb{E}_P[Y_M(TU)] = \mathbb{E}_P[Y_M(U)]$; the covariance identity is otherwise unchanged. Apply the same identity to the deterministic continuation at each fixed history under conditional preservation, then use iterated expectation. □

Remark (The statistical reading, and what is measured). For a PRNG stream the shifted tail starts from its advanced internal state. This mechanical fact implies neither a fresh-seed trajectory law nor independence, Gaussian residuals, or reduced endpoint covariance. Even under an ideal i.i.d. future stream and a fixed history-measurable displacement, the retained training state and history remain shared; conditional measure preservation can preserve a continuation's marginal law without making the paired continuations independent. Adaptive displacement requires its own invariance check. Realized divergence and its onset are trace measurements, not consequences of changed intermediate values alone. Endpoint magnitude, sign, and covariance are empirical properties; none is derived from cursor arithmetic.

C.3 Proposition 2: harvest size

Proposition (Harvest size)**.** *For a finite vector of integrable paired gaps $D = (D_1, \dots, D_m)$, let $\mu_j = \mathbb{E}D_j$, $\bar{\mu} = m^{-1}\sum_j \mu_j$, and let J be a fixed tie-broken maximizer of D_j. For arbitrary means and dependence,*

$$\mathcal{O} := \mathbb{E}[D_J - \mu_J] \geq \max_j \mu_j - \mathbb{E}\mu_J \geq 0, \qquad \mathbb{E}H(D) = \mathbb{E}[\mu_J - \bar{\mu}] + \mathcal{O},$$

where $H(D) = \max_j D_j - m^{-1}\sum_j D_j$. The mean-selection term need not be positive. Common additive components cancel from H. If $D_j = a + C + \sigma Z_j$, with $\sigma \geq 0$, integrable common C and i.i.d. standard normals Z_j, then $\mathbb{E}H = \mathcal{O} = c_m\sigma$ and $c_m = \mathbb{E}\max_j Z_j$. Independence of C from the Z_j is unnecessary.

Proof. Since $D_J = \max_j D_j$, $\mathbb{E}D_J \geq \max_j \mathbb{E}D_j$; subtracting $\mathbb{E}\mu_J \leq \max_j \mu_j$ proves the inequalities. Adding and subtracting μ_J in $D_J - \bar{D}$ proves the decomposition. For example, deterministic $D = (0, 3, 6)$ gives $H = 3$ but $\mathcal{O} = 0$: uplift need not be noise optimism. In the Gaussian special case, $H(D) = \sigma(\max_j Z_j - \bar{Z})$ pathwise and all μ_j are equal, so $\mathbb{E}H = \mathcal{O} = c_m\sigma$. The maximum density is $m\varphi(x)\Phi(x)^{m-1}$, hence

$$c_m = m\int_{-\infty}^{\infty} x\,\varphi(x)\,\Phi(x)^{m-1}\,dx.$$

Numerical quadrature gives the constants in Table 8, in particular $c_6 = 1.2672$ and $c_7 = 1.3522$. □

Remark (Offsets, dependence, and the scale being estimated). Let $P = I - \mathbf{1}\mathbf{1}^\top/m$ and let α be deterministic. Since $H(x) = \max_j(Px)_j$ and the maximum changes by between the smallest and largest added coordinate,

$$\min_j(P\alpha)_j \leq \mathbb{E}H(\alpha + \varepsilon) - \mathbb{E}H(\varepsilon) \leq \max_j(P\alpha)_j.$$

Thus offsets need not leave a Gaussian benchmark or its ratio unchanged. For $m \geq 2$ and a square-integrable vector D with mean μ and covariance Σ, the within-grid sample variance $S^2 = \|PD\|^2/(m-1)$ satisfies

$$\mathbb{E}S^2 = \frac{\|P\mu\|^2 + \operatorname{tr}(P\Sigma)}{m-1}.$$

Indeed $\mathbb{E}\|PD\|^2 = \|P\mu\|^2 + \operatorname{tr}(P\Sigma P)$ and $P^2 = P$. The spread therefore mixes mean contrasts and stochastic contrasts; offsets do not guarantee a particular change in a realized maximum or observed-to-predicted ratio.

For equal-mean equicorrelated Gaussian residuals $\varepsilon_j = \sigma(\sqrt{\rho}Z_0 + \sqrt{1-\rho}Z_j)$, $0 \leq \rho < 1$, the common factor cancels and $\mathbb{E}H = c_m\sigma\sqrt{1-\rho}$. Here σ is the *marginal* residual scale; a within-grid SD already estimates the contrast scale $\sigma\sqrt{1-\rho}$. Multiplying it by another $\sqrt{1-\rho}$ would double-correct correlation. Under equal means and i.i.d. normal residual coordinates (or this common-factor extension), $\mathbb{E}S = c_4(m)\sigma_{\text{contrast}}$, where

$$c_4(m) = \sqrt{\frac{2}{m-1}}\,\frac{\Gamma(m/2)}{\Gamma((m-1)/2)}.$$

This normal-theory correction (distinct from the maximum constant c_4) does not generally hold with unequal means or arbitrary dependence. Consequently $c_m\bar{\sigma}$ is an in-grid descriptive Gaussian benchmark, not an independent prediction or a universal pricing law.

m	2	3	4	5	6	7
c_m (exact)	0.5642	0.8463	1.0294	1.1630	1.2672	1.3522
$\sqrt{2\ln m}$	1.1774	1.4823	1.6651	1.7941	1.8930	1.9728
overshoot	+108.7%	+75.2%	+61.8%	+54.3%	+49.4%	+45.9%

Table 8: The asymptotic $\sqrt{2\ln m}$ is unusable at the m of real reporting practices: exact expected maxima of m standard normals (quadrature of the expression in the proof) against the leading-order asymptotic and its relative overshoot.

Remark (Validation selection and held-out reporting). Let J maximize validation scores with fixed tie breaking and let integrable T_j be the corresponding test quantities, with $\mu_j^T = \mathbb{E}T_j$. Candidate-specific centering gives the exact identity

$$\mathcal{O}_T := \mathbb{E}[T_J - \mu_J^T] = \sum_{j=1}^{m} \mathrm{Cov}(\mathbf{1}\{J = j\}, T_j).$$

Expand T_J with selection indicators to prove the identity. Its sign is unrestricted: positive validation–test Pearson correlation does not fix these selection-indicator covariances. For a finite counterexample, take x uniform on $(-2, -1, 1, 2)$, validation scores $(x, -x)$, and test scores $(t, -t)$ with respective $t = (-10, 15, -15, 10)$. Both candidate correlations are positive, but all test means vanish and $\mathcal{O}_T = -2.5$. An observed selected test delta from a common baseline is not this optimism unless the same candidate-specific mean reference is used. Stronger transfer models may yield positive benchmarks; correlation alone is not such a model.

Remark (Why not $\sqrt{2\ln m}$). Extreme-value asymptotics give

$$c_m = \sqrt{2\ln m} \; - \; \frac{\ln\ln m + \ln 4\pi}{2\sqrt{2\ln m}} \; + \; \frac{\gamma}{\sqrt{2\ln m}} \; + \; o\big((\ln m)^{-1/2}\big),$$

with γ the Euler–Mascheroni constant. The leading term alone overshoots the exact constant by +49.4% at $m = 6$ and +45.9% at $m = 7$ (Table 8), the negative second-order term staying non-negligible until $\ln\ln m \gg 1$. A prediction built on $\sqrt{2\ln m}$ would have overshot the measurement by half, so the data are tested against the exact constants and the asymptotic form is kept only as growth-law language.

Remark (Common components and shared baselines). The invariance clause is what makes the statistic measurable here: all placements within a seed share one baseline run, so the baseline's realization enters every paired gap Δ_j as a common component and cancels exactly in max-minus-mean, as does any common probe main effect. What survives is the placement contrast, which may contain unequal means as well as noise. Proposition 1(ii) does not specify its law.

Remark (Model premises and reviewer-requested sensitivity). Independence and approximate normality are modeling premises, not theorems. The nested growth curve reuses the placement cells used to estimate $\bar{\sigma}$, so it is an in-grid shape check, not independent prediction. Exact quadrature gives the descriptive comparison 4.0107 points of uplift versus $c_6\bar{\sigma} = 4.1203$; the normal-specific $c_4(m)$ correction gives 4.3301. The uncorrected ratios for shortened, realistic-evaluation, full-best and full-final endpoints are 0.9734, 1.1463, 1.0075, 1.0034; the corrected ratios are 0.9262, 1.0775, 0.9282, 0.9245. Best and final share three seeds and are not independent replications.

The reviewer-requested bootstrap jointly resamples each complete seed row in numerator and denominator, enumerating all $5^5 = 3{,}125$ or $3^3 = 27$ ordered samples. Corrected percentile intervals are respectively $[0.7003, 1.1155]$, $[0.8945, 1.2009]$, $[0.8665, 0.9776]$ and $[0.6611, 1.2452]$. The full-best interval excludes one, but these discrete three-seed resamples do not establish calibrated coverage or reliable model rejection. Unequal placement means and contrast covariance can enter the fitted scale: the empirical placement covariance and row-centered contrast covariance have rank four with five seeds and rank two with three seeds, and do not validate an unrestricted covariance model.

The original constant-only Gaussian/t_3 curves use 200,000 Monte Carlo draws; their reproducibility does not establish iid residuals. The new full-procedure sensitivity instead generates 100,000 complete matrices under normal and variance-matched t_3 models, re-estimating every SD and the ratio of means in each replicate

(fixed seed 20260908). At 5×6, the normal-corrected ratio has means 0.9997 and 1.0177 and respective sampling-reference intervals $[0.8087, 1.1979]$ and $[0.7251, 1.3014]$; at 3×4 the means are 1.0001 and 1.0008. Using observed $s_s/c_4(m)$ as fixed seed-specific scales gives similarly overlapping references. Thus constant-only t_3 underprediction does not discriminate the residual distributions once scale estimation is included. The normal $c_4(m)$ multiplier is part of the same statistic in both simulations, not a valid t_3 unbiased-scale correction. These are simulation sensitivities, not additional training evidence.

Remark (The cross-seed prediction, and what it refutes). The historical cross-seed implementations use different scales: `scripts/prop2_fit.py` flattens the remaining rows before calculating one SD, whereas `figs/make_fig_r1_crossseed.py` averages their within-row SDs. The former mixes between-seed means into the scale; both are reported in the canonical reanalysis. For the latter, predicting the fifth seed's max-minus-mean as $c_6\bar{\sigma}$ *fails at the individual-seed level*. Predicted versus observed (points): 3.95/3.00, 4.38/2.71, 3.87/5.39, 4.71/1.63, 3.70/7.32: mean absolute error 2.17, 54% of the 4.01 being predicted, with the two series *anti*-correlated ($r = -0.8672$); mean signed error is $+0.11$. The flattened-row alternative has MAE 2.2441 and $r = -0.9683$; the $c_4(m)$-corrected within-row alternative has MAE 2.2210. Mean-other-SD predictions are affine decreasing functions of held-out SD, so their correlation with harvest is the negative of the *observed* SD–harvest correlation, not necessarily negative in every population or dataset. Substituting each seed's own SD reduces MAE to 0.8256, but uses the same outcomes as the measured harvest and is not a held-out validation. These results document failed seed-level scale transport; they neither independently validate nor isolate the Gaussian constant as the sole correct part of the model. We make no per-run prediction from the aggregate fit.

Remark (Instantiation constants and statistic definitions). m is fixed by design, not fitted. The multi-seed campaign uses $m = 6$ placements per seed (c_6; numbers above). The single-seed official-code demonstration uses $m = 7$ (c_7): its selection max-minus-mean over the 8-realization set (7 placements plus the no-probe run) is $+2.88$ ($32.60 - 29.72$), a single-seed realized value rather than a mean, and $+3.06$ over the 7 placements alone (an earlier 6-placement mid-campaign figure of $+3.24$ is superseded). In the same set the honest average placement effect is -1.44 and the worst single placement -8.68 (reproduced exactly at Appendix D's floor resolution under a different concurrency condition): the probe hurts on average. The selected maximum's uplift over the placement average is not an identified noise-only gain. The multi-seed grid these constants are read off is Figure 7.

C.4 Proposition 3: meta-signal scale

Proposition 3 (Meta-signal scale)**.** *For a single SGD virtual step $\theta'(w) = \theta - \eta\nabla_\theta \sum_i w_i \ell_i(\theta)$ and a meta-objective $L_{\text{meta}}(\theta')$ with no direct w-dependence, $\partial L_{\text{meta}}/\partial w_i = -\eta \langle \nabla L_{\text{meta}}(\theta'), \nabla \ell_i(\theta)\rangle$ exactly; the meta-signal is $O(\eta)$ for bounded gradients, while any η-independent term competing for the same weights is unmoved. If, in addition, ∇L_{meta} is locally Lipschitz near fixed θ and the weighted update direction is uniformly bounded as $\eta \to 0$ (in particular, for fixed w), the direction field has an $O(\eta)$ perturbation within that neighborhood. For a nonzero limiting direction and a nonzero normalization denominator, a post-map $w = N(\text{signal})$ removes the explicit multiplicative η scale iff N is positively homogeneous of degree zero; it does not remove direction changes, and the zero-direction case is outside this claim.*

Proof. The virtual step is affine in w with a w-independent Jacobian: $\theta'(w) = \theta - \eta \sum_j w_j \nabla_\theta \ell_j(\theta)$, where θ and the per-example gradients $\nabla_\theta \ell_j(\theta)$ are evaluated at the unperturbed point and do not depend on w; hence $\partial\theta'/\partial w_i = -\eta\, \nabla_\theta \ell_i(\theta)$ exactly. Because L_{meta} has no direct w-dependence, the chain rule gives, with no expansion in η,

$$\frac{\partial L_{\text{meta}}(\theta'(w))}{\partial w_i} \;=\; \Big\langle \nabla_{\theta'} L_{\text{meta}}(\theta'),\, \frac{\partial \theta'}{\partial w_i} \Big\rangle \;=\; -\eta \,\big\langle \nabla L_{\text{meta}}(\theta'),\, \nabla \ell_i(\theta) \big\rangle.$$

Both stated conditions are load-bearing. Multiple inner steps or an adaptive inner update are outside this identity: their exact Jacobians and leading-order scales require separate derivation. A direct w-dependence of the meta-objective adds an η-free term to $\partial L_{\text{meta}}/\partial w_i$, exactly the competition mode the proposition prices. For scale, $|\partial L_{\text{meta}}/\partial w_i| \le \eta\, \|\nabla L_{\text{meta}}(\theta')\|\, \|\nabla \ell_i(\theta)\|$, so with gradients bounded on the visited set the meta-signal is $O(\eta)$, an upper envelope rather than a guaranteed magnitude, since the inner product can

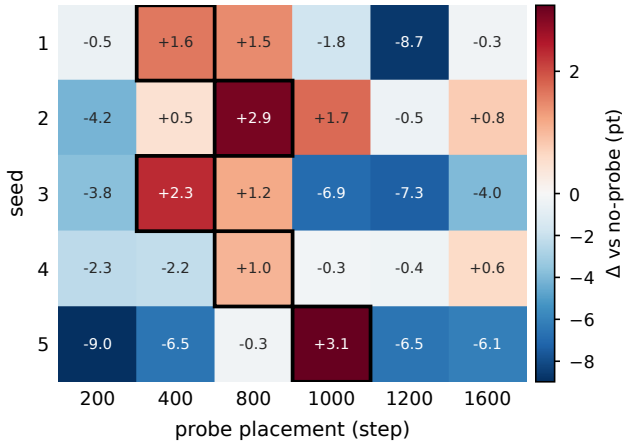


Figure 7: **The harvest grid behind the Proposition 2 fit:** paired deltas (points) for 5 seeds × 6 probe placements, diverging scale centered at 0, per-seed maximum outlined. The grid spans −8.98 to +3.08 points with honest mean −1.82; the outlined maxima (+1.62, +2.92, +2.30, +1.04, +3.08) are what per-seed selection reports, and the −8.68 cell (seed 1, step 1200) is the floor-verified outlier of Table 11. The maxima land on three different placements. A legacy placement test gives $p = 0.049$ under within-seed permutation, conditional on exchanging fixed training positions being a valid null. The general part of Proposition 2 allows unequal means, but its Gaussian special case does not absorb them: σ is estimated as within-seed spread *across* placements and mixes offset and noise contrasts, without a guaranteed matching change in the observed maximum. In simulation, treating the observed column means as real offsets moves the *expected* observed-to-predicted ratio from 1.00 to 1.04 (a sensitivity check, not one of the campaign ratios of Figure 4b), and the observed harvest sits at the 53rd percentile of the null in which no placement effect exists at all.

vanish, which is why the statement is $O(\eta)$ and not $\Theta(\eta)$. An input to the same consumer carrying no factor of η is unmoved as $\eta \to 0$; what that costs a controller in practice is the subject of Section C.5.

Normalization escape. Write the signal field as $g_\eta = -\eta\, h_\eta$ with $h_{\eta,i} = \langle \nabla L_{\text{meta}}(\theta - \eta G w),\, \nabla \ell_i(\theta) \rangle$ and $Gw = \sum_j w_j \nabla \ell_j(\theta)$; then, for fixed θ, uniformly bounded Gw and ∇L_{meta} locally Lipschitz near θ, $h_\eta = h_0 + O(\eta)$, so the η-dependence factors into an exact multiplicative scale and an $O(\eta)$ perturbation of the direction. Say a post-map N *removes the scale* if $N(\eta h) = N(h)$ for every signal h in its domain and every $\eta > 0$; we prove this holds iff N is positively homogeneous of degree zero, i.e. $N(c\,x) = N(x)$ for all $c > 0$ and x in the domain. ($\Leftarrow$) $N(g_\eta) = N\big(\eta \cdot (-h_\eta)\big) = N(-h_\eta)$: the scale is removed exactly, and residual η-dependence enters only through the $O(\eta)$ rotation of the direction. ($\Rightarrow$) Fix x in the domain and $c > 0$; apply the removal property with $h = x$ at $\eta = c$ and at $\eta = 1$: $N(cx) = N(x)$. □

Remark (Sum-to-one is not the property). $\mathrm{softmax}(c\,g)$ tends to the uniform vector as $c \to 0^+$ for every non-constant g: softmax sums to one, is invariant to additive shifts, and is not positively homogeneous of degree zero. Fed an η-scaled signal, a softmax-normalized controller's weight contrasts vanish as $\eta \to 0$, but need not be zero at finite η. What rescues the rectify-then-normalize map used by L2RW is homogeneity: $N(g) = \mathrm{ReLU}(g)/\langle \mathbf{1}, \mathrm{ReLU}(g) \rangle$ is a ratio of two positively 1-homogeneous maps, hence 0-homogeneous wherever the denominator is nonzero.

Remark (The zero-guard is load-bearing). Both audited L2RW implementations guard the normalization at exactly zero: the TF release increments the denominator only when it equals 0.0 (`reweight.py:150-153`), and the PyTorch notebook branches on `norm_c != 0`, so the guard fires only when the rectified signal is exactly zero, i.e. when no example carries positive signal, and the escape is intact for every nonzero signal. A threshold variant (normalize only when $\sum_i \mathrm{ReLU}(g_i) > \delta$ for fixed $\delta > 0$) bypasses scale cancellation whenever the rectified sum is at or below δ: the raw η-scaled weights pass through. This branch does not itself imply zero action or establish that every batch crosses the threshold.

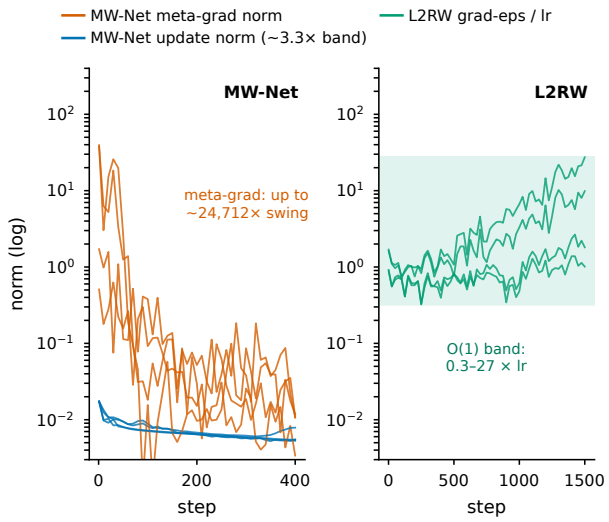


Figure 8: **Signal-versus-update decoupling in the two re-executed meta-reweighting systems (four runs each: noise $\{0, 0.4\}\times$ seeds $\{1, 2\}$).** Left: Meta-Weight-Net's raw meta-gradient norm (vermillion) decays 150×–3,700× from first to last measurement across the four runs, and its full within-run max / min swing reaches ∼24,700×, while the induced v-net update norm (blue) stays inside a 3.3× band throughout. Right: L2RW's raw meta-gradient divided by the learning rate lies between 0.32 and 27 (shaded), consistent with raw $O(\eta)$ scaling. The separate rectify-then-normalize map removes the explicit scale by 0-homogeneity (Proposition 3); applied weights are not plotted.

C.5 The dead-meta-gradient taxonomy (a second, appendix-scoped diagnosis)

This diagnosis is deliberately appendix-scoped. The main text carries the paper's contribution: a condition, a prevalence audit, byte-level witnesses, a harvest law, and a repair. What follows is a second, narrower diagnosis of the same method family, established analytically and instrumented in three systems (Figure 8) rather than audited at scale, and reported because it lands on the same victims.

Proposition 3's $O(\eta)$ scale can become negligible in *competition*: the relevant magnitude ratio compares the η-scaled signal with whatever else reaches the same weights. An η-independent input of comparable or larger size (a regularizer, a fixed-scale update) can dominate a signal scaled by $\eta \approx 10^{-5}$–10^{-3}, but this does not imply exactly zero controller action. The outcome depends on the consumer and the learning-rate schedule, not merely on the presence of a competing term. Three named escapes can mitigate this sensitivity: a degree-0-homogeneous weight map cancels the explicit multiplicative scale, while an independent adaptive optimizer can reduce (but, with the usual $\epsilon > 0$, does not exactly remove) sensitivity to gradient magnitude. A signal that is η-independent by construction (a real validation-metric difference between materialized candidate models, the choice our constructive endpoint makes) sidesteps the competition entirely (Table 9). The dead regime is not hypothetical: our own predecessor system was the specimen, its signal $\sim 10^{-6}$ against a $\sim 10^{-4}$ regularizer, every learned amplitude pinned at initialization and its "filtering" map identically the identity for entire runs.

The design rule is to check the signal's scale at its consumer: an η-independent quantity or a scale-invariant map can avoid explicit lr-scale competition, while adaptive optimization can mitigate it. An lr-scaled signal may become negligible relative to a fixed-scale competing term as the learning rate decays; exact inactivity requires additional consumer behavior or measurement. This is why the two diagnoses fall on the same victims: the virtual-step probe that makes a controller's signal scale suspect is also the most RNG-hungry pattern in our audit, so the method family that most needs trustworthy paired evaluation is the one whose default implementation most reliably destroys it.

Class	Signal path	In-system evidence
Dead	lr-scaled virtual step; consumer shares parameters with an η-independent regularizer	in-house predecessor: signal $\sim 10^{-6}$ vs. own regularizer $\sim 10^{-4}$; weights pinned ($\pm 0.5\%$); filtered/raw $\equiv 1.0000$
Rescued by construction	virtual step, then rectify-and-normalize weight map (degree-0 homogeneous)	L2RW PyTorch reimplementation, driven unmodified: lr-normalized raw gradient $\|g_\epsilon\|/\eta = O(1)$ throughout training (0.3–27 across four runs)
Decoupled by construction	true meta-gradient consumed by an independent Adam optimizer	MW-Net official, driven unmodified: update norm holds a $\sim 3.3\times$ band while the raw meta-gradient swings $150\times$–$3{,}700\times$ first-to-last over the same window across the four runs

Table 9: A structural taxonomy of meta-controller signal paths, classified by the signal's route to its consumer. The claim is signal *scale reaching the consumer*, not end-task efficacy; per-row evidence is in native units because raw magnitudes are confounded by η, phase, and norm convention.

We state the scope plainly, because the analysis is more general than the measurement. Proposition 3 fixes the signal's scale exactly and for every method of this form; whether that scale is *fatal* is a question about a magnitude ratio, and we measured that ratio in three instrumented systems rather than mapping it across the method family, across learning-rate regimes, or across competing-term sizes. This is therefore a theoretically grounded failure mode illustrated in three systems, not a generic fate of virtual-step controllers. The proportions matter and we state them: exactly one of the three is dead, and it is our own predecessor; both *released* systems we instrumented escape, L2RW by a degree-0-homogeneous weight map and Meta-Weight-Net by an independent adaptive optimizer. Nothing here claims that any published method fails at its end task, and those two escapes are evidence that the pattern is avoidable. How often it is avoided in the wild is not something three instrumented systems can say.

C.6 Corollary: the certified no-op

Corollary (Certified no-op)**.** *Under the deterministic pairing protocol of Appendix D and state-pure, stream-isolated candidate maintenance, champion = raw implies that the served model state (parameters and buffers) is bitwise identical to the paired baseline endpoint, and hence every deterministic reported metric delta is exactly zero. Conversely, a bitwise baseline identity implies champion = raw only under the additional, checked premise that every non-raw candidate differs from raw in at least one parameter bit. Candidate collisions (for example an EMA landing exactly on the endpoint) are not excluded by training having moved and must be treated as exact-collision events rather than assumed impossible.*

Proof. State and stream isolation preserves every training consumption, so Proposition 1(i) gives $\theta_M^{\text{raw}} \equiv \theta_B$ for the full endpoint state. If champion = raw, the served state equals θ_M^{raw} and hence equals θ_B; deterministic evaluation on a fixed set then gives exact metric equality. For the converse, if the served state equals θ_B and champion is non-raw, the checked distinct-candidate premise says that served state differs from $\theta_M^{\text{raw}} \equiv \theta_B$, a contradiction. □

Remark (Resolution, and the falsification lever)*.* The ($\Leftarrow$) direction is a statement about weight-level identity. At metric resolution it is not invertible (two different models can collide at $1/n$ accuracy resolution), so the certificate is operational rather than metric-inferred: the controller logs its champion. A certificate requires the state-purity and isolation premises, not just a raw champion and a zero metric floor. A nonzero delta in a champion-raw cell falsifies the premise conjunction but does not uniquely identify stream displacement: hidden state mutation or an untested nondeterministic path can also account for it. Across the evaluated campaigns every champion-raw cell reported exactly zero (18/18 tuned, Table 10; 6/6 under-tuned canaries);

Tuned regime	Guard Δ	Exact zeros	LAWA
NLP, linear decay (3 tasks × 3 seeds)	**0.00**	9/9	−1.44 (6/9 neg)
Vision SGD, clean	**0.00**	5/5	−0.11 (4/5 neg)
Vision SGD, noise30	**0.00**	5/5	−1.16 (5/5 neg)

Table 10: Observational guard registry, not a count of state-level certificates. The reported delta is zero in all 19 cells and champion = raw in 18. The eight NLP raw-champion cells were historically described as hash-verified, but the recovered per-cell logs contain no independently checkable weight hashes and have no repeat floors; they are excluded from certificate counts. The ten vision cells have metric-level evidence only (Figure 5); the nineteenth is an exact EMA–baseline holdout tie logged as a promotion, while unguarded fixed-window averaging (LAWA) nets harm on the identical protocol.

the count includes metric-only cells without an independent floor, so it is not a bitwise certificate. The two other exact zeros are EMA holdout ties logged as promotions rather than certificates.

C.7 Lemma: margin calibration under paired probe noise

This auxiliary selection analysis is not needed for the pairing condition or the no-op implication. We retain its proof and the candidate-comparison figures to document the guarded endpoint separately from the main diagnosis.

Lemma 2 (Margin calibration)**.** *At a selection event, a candidate is scored against raw on the same probe split. Let $D_i \in \{-1, 0, 1\}$ be the paired correctness difference, $D = \sum_i \mathbf{1}\{D_i \neq 0\}$ the discordant count, and $W = \sum_i \mathbf{1}\{D_i = 1\}$ the candidate-win count. Assume only that the probe examples are i.i.d. for this fixed candidate. If its population paired effect satisfies $\Delta = \Pr(D_i = 1) - \Pr(D_i = -1) \leq 0$, then conditional on $D = d$, $W \sim \mathrm{Bin}(d, \pi)$ with $\pi \leq 1/2$. Consequently, any promotion threshold $w_\alpha(d)$ satisfying*

$$\Pr\{\mathrm{Bin}(d, 1/2) \geq w_\alpha(d)\} \leq \alpha$$

has conditional and unconditional false-promotion probability at most α. For C candidates at one event, use α/C; for K planned events, allocate levels $\alpha_{e,c}$ with $\sum_{e,c} \alpha_{e,c} \leq \alpha$. This requires a valid marginal test for every indexed comparison: for example, all compared model pairs fixed independently of the reused probe, or fresh i.i.d. probe examples independent of each adaptively chosen pair conditional on its past. No independence among such valid tests is needed for the union bound. Alpha allocation alone cannot make adaptively reused probe data valid, even for a finite number of comparisons.

Proof. Write $q = \Pr(D_i = 1)$ and $r = \Pr(D_i = -1)$. Conditional on discordance, the win probability is $\pi = q/(q + r)$ (unless $q + r = 0$), and $\Delta = q - r \leq 0$ implies $\pi \leq 1/2$. The binomial tail is monotone in its success probability, so the stated threshold is conservative conditionally on every $D = d$; averaging over D proves the unconditional bound. Bonferroni gives the displayed allocations without any cross-event independence. □

Remark (Scope of the margin result). The conditional rule avoids a guessed variance reduction or a population accuracy replaced by its same-probe estimate. It is a prospective calibration option, not a retrospective error guarantee for the historical fixed-margin campaigns. State-pure RNG isolation does not make an adaptively reused validation set statistically fresh. The no-op corollary is a separate pathwise statement and does not require this false-promotion guarantee.

Remark (Small n, conservatism, and the realized error profile). For a small discordant count D, normal-tail calibration can be inaccurate; the conditional binomial test handles that discreteness exactly under its fixed-pair sampling premise. The guard inherits the conservative side by design: strict inequality plus tie-to-raw resolution biases errors toward keeping raw, whose cost is exactly zero by Corollary 1 under its premises. The historical campaign record has no negative observed holdout deltas in 19 tuned cells (one exact tie was promoted), and one realized selection loss in 35 under-tuned cells (−3.12; the candidate cleared its margin on the probe and failed to generalize to the holdout, the residual risk a probe margin cannot price, reported rather than averaged away). What the guard pays out where it does act, against LAWA and against

an unconditional always-EMA policy, is Figures 12 and 13; the cell-by-cell record and its largest single rescue are Figures 14 and 15, and the participation-ratio detector that gates the candidates is Figure 18.

D The Deterministic Pairing Protocol and the Repeat-Floor Registry

D.1 Protocol

1. **Deterministic kernels.** Every paired run uses deterministic kernels:
   - `torch.use_deterministic_algorithms(True)` (which also constrains cuDNN to deterministic algorithms; with `warn_only=True` in the DivideMix and BHN witnesses);
   - `CUBLAS_WORKSPACE_CONFIG=":4096:8"`, exported before CUDA context creation, as cuBLAS determinism requires (the NLP harness: see below).

   The nanoGPT campaign additionally sets `torch.backends.cuda.matmul.allow_tf32 = False` and `torch.backends.cudnn.allow_tf32 = False`; the other campaigns leave TF32 at the PyTorch defaults, and their per-path repeat floors (item 3) are measured under those settings. The NLP harness reaches deterministic execution through `transformers' enable_full_determinism(seed)`, which sets `CUBLAS_WORKSPACE_CONFIG=":16:8"` itself. Proposition 1 is stated under deterministic execution, and the second layer is real: in the under-tuned CIFAR-100 protocol, same-seed cross-arm bifurcation from nondeterministic kernels alone reached 11.7 points (Figure 9).
2. **Stream isolation for every internal consumer.** Every monitor statistic and controller probe executes inside `torch.random.fork_rng` with the active accelerator listed (`devices=[device]` under CUDA, `devices=[]` on CPU), which forks the CPU stream unconditionally and the listed device streams (Appendix B.2): the fork-restore discipline of Proposition 1(i). Probes are state-pure in the sense of Appendix C.1: eval-mode forwards, no optimizer interaction, candidates kept as CPU-resident snapshots.
3. **The repeat-floor procedure.** Each materially distinct execution path receives its own within-path repeat: both arms are rerun when their kernel, control flow, precision, stochastic consumer, or probe differs, unless a shared path has been explicitly established. The *repeat-floor* is the largest difference observed at the resolution of the claim: metric-exact supports only a metric claim, weight-exact supports only the recorded parameters and buffers, and full-state or trajectory claims require hashes of the corresponding optimizer, scheduler/scaler, controller, RNG/cursor, and checkpoint states. A zero floor is evidence on the tested path, not a universal determinism proof; a nonzero floor voids the family's paired reading. The floor converts "we set seeds" from a hope into a measurement.
4. **Atomic result writes and completeness audits.** Campaign outputs are written to a temporary file and moved into place by atomic rename (`tmp` + `os.replace`), so an interrupted run cannot leave a truncated file that aggregation would read as complete; line counts audit completeness (30/30 run files, zero holes, under-tuned NLP campaign).
5. **Regime hygiene.** Deltas from non-deterministic legacy campaigns never enter deterministic tables, and where a legacy figure did not survive re-measurement only the deterministic value is reported (CIFAR-100 clean: legacy +8.59 withheld, deterministic re-measurement +4.86).

D.2 Per-campaign repeat-floor registry

Two reading notes on Table 11. First, floors are path-specific: a floor checks repeatability of the recorded observables for the tested configuration and stack, not cross-arm pairing (Appendix E). nanoGPT makes the point concretely: a canonical-configuration rerun (1,500 iterations, default CUDA kernels) fails its own repeat floor, identical repeats differing by $\max|\Delta W| \approx 9\times10^{-2}$, so no paired claim is made there, and the historical witness is registered at the 200-iteration configuration whose floors are exactly zero. Second, the recorded Meta-Weight-Net flip and harvest demonstrations are accuracy-exact rather than weight-level: final accuracies at 2×10^{-4} resolution bound those floors at the same resolution as the reported effects (the 1.78

Experiment family	Floor evidence	Comparison resolution
HuggingFace probe experiments (RoBERTa-base; RTE, MRPC; five-arm, ten-pair effect size, full-length)	16 floor files across the campaigns, all exactly 0.0: each repeat arm verified bitwise identical to its reference	weight-level max $\lvert\Delta W\rvert$
NLP tuned, gate B (RTE/MRPC/CoLA × 3 seeds, linear decay)	*no independent rerun*: 8/9 cells champion = raw with Δ exactly 0.00; the ninth (MRPC seed 2) an exact EMA–baseline holdout tie at $\Delta = 0.00$	metric-exact; not a rerun
NLP under-tuned (3 tasks × 5 seeds)	30/30 run files complete (line-count audit, zero holes); *no independent rerun*: the 6 champion-raw cells are observations only, all at Δ exactly 0.00 (a seventh exact zero is an EMA-champion holdout tie)	final metric, exact; not a rerun
Vision tuned, deterministic (CIFAR-10, SGD cosine + augmentation; clean and noise30)	repeat floor exactly 0.0; 10/10 cells report the guard at exactly 0.00 (float-equal finals)	final metric, exact
Vision under-tuned, deterministic (CIFAR-10/100, constant lr, no augmentation; clean and noise30)	`tmp` + atomic-rename write protocol; whole-pipeline integrity rerun: seed 1, all three arms re-run from scratch reproduce the archived finals bitwise (0.802444 / 0.824222 / 0.783333)	bitwise on stored finals
Meta-Weight-Net official-code flip and harvest demonstrations	repeat arms exact at final-accuracy resolution; the outlier placement (−8.68) independently re-run and exactly equal (22.30 ≡ 22.30) under a different GPU concurrency condition; five-seed registration rerun reproduces the family bitwise (final model and meta-net SHA-256 identical, seeds 1–5), and the `fork_rng`-isolated probe lands bitwise on the never-probed baseline in 5/5 seeds	accuracy-exact 2×10^{-4} on the recorded demonstrations; weight-level SHA-256 on the registration rerun
nanoGPT four-arm witness (200 iterations, fp32, `shakespeare_char`)	both same-configuration floors exactly 0: A ≡ B (no eval) and C ≡ D (with eval); the eval-versus-no-eval contrast, A vs. C, is max $\lvert\Delta W\rvert = 3.3\times10^{-2}$	weight-level max $\lvert\Delta W\rvert$
nanoGPT calibration/held-out campaign (2,000 updates, deterministic FP32)	32 seeds, each with baseline, five shared and five isolated paths; each path run twice (704 main runs). All 352 repeat pairs are exact, including zero fixed-panel loss difference. See Appendix D.5 for recorded state components.	endpoint and sparse state checkpoints; fixed-panel loss; not every update
DivideMix four-arm witness (12 epochs, 50% symmetric noise, CIFAR-10)	both same-configuration floors exact at final-weight SHA-256 (stock-GMM pair identical; pinned-GMM pair identical); the cross-configuration contrast is nonzero (final accuracies 0.73 apart) with warm-up accuracy logs byte-identical across all four arms	weight-level SHA-256
BHN four-arm witness (12 epochs, CIFAR-10, official ResNet-34)	both same-configuration floors exact at final-weight SHA-256, matching per epoch (isolated-test-loader pair identical; stock pair identical); the cross-configuration contrast is nonzero (final accuracies 1.93 points apart) with per-epoch weight hashes byte-identical through epoch 1 and diverging exactly at epoch 2, the first epoch whose training draws follow the differing probe	weight-level SHA-256, per-epoch
E1 validation-to-test bridge (20 seeds × 6 arms)	seed-101 baseline and repeat identical in model and meta-network SHA-256; 120/120 registered arms complete, with no partial files	weight-level SHA-256; complete-arm audit

Table 11: Repeat-floor registry with explicit resolution and coverage limits. Reported zero floors concern only the listed rerun paths; weight hashes do not cover complete execution state, and neither weight nor metric equality proves a cross-arm trajectory guarantee. The NLP rows have no independent rerun and are excluded from certificate counts rather than inferred through Corollary 1. E1 has a baseline-only floor, not repeat coverage for all placement paths. The new 2,000-update nanoGPT campaign repeats all eleven paths per seed; its sparse checks do not imply every-update trajectory capture. The revised protocol requires each materially distinct path to be repeated at the claimed resolution.

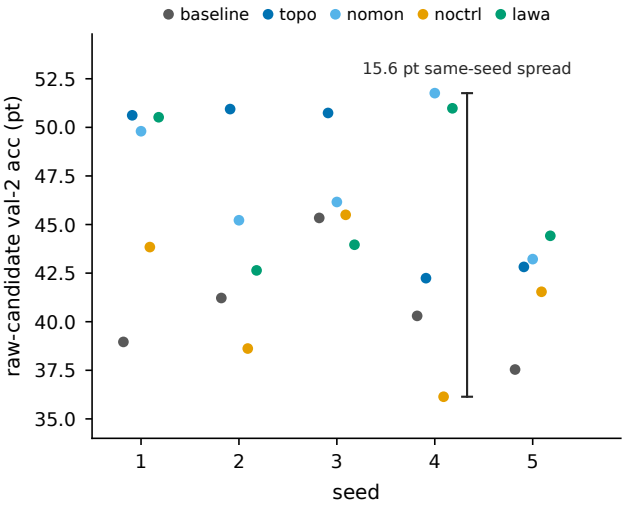


Figure 9: **Kernel nondeterminism alone breaks same-seed pairing (legacy non-deterministic CIFAR-100 stratified campaign, clean labels):** every arm trains the same recipe from the same seed, yet the raw candidates spread by 6.8–15.6 points within a seed. Dots are the five arms per seed; the bracket marks the widest five-arm max − min spread (15.6 points, seed 4). The main text's 11.7-point figure is the pairwise topo-versus-baseline gap on seed 1 (50.62 vs. 38.96), a two-arm special case of the same bifurcation; the deterministic protocol above eliminates it (repeat floors exactly 0).

and 1.16 point gap movements on seeds 1 and 2, in opposite directions, drawn in Figure 10, are 89× and 58× that resolution). A five-seed registration rerun of the same configuration closes the resolution gap at family level: rerun versus rerun, final model and meta-net states are SHA-256 identical in 5/5 seeds, and the `fork_rng`-wrapped probe (verified to fire) lands bitwise on the never-probed baseline, the repair protocol's isolation clause verified on the witness codebase itself.

D.3 E1: the validation-to-test bridge

Design. The realistic-evaluation harvest selects placements on the reported metric itself. E1 tests the stricter practical bridge: for each of 20 seeds, a post-training clean validation split of 4,000 examples selects among the never-probed baseline and five monitoring-evaluation placements (steps 400, 1,400, 2,700, 4,000, 5,200), while the untouched official CIFAR-10 test split is read once, after training. Ties favor the baseline, then the earlier placement; both holdout evaluations are sequential and consume no RNG. All 120 registered arms completed, and a same-machine repeat of the seed-101 baseline is identical in model and meta-network SHA-256.

Frozen readout and separate reviewer-requested analysis. The frozen comparator reports the primary selected-test mean +0.1245 with its original seed-level t interval $[-0.0406, 0.2896]$. It selects a placement for 17/20 seeds; the three baseline champions contribute exact zeros, and all other zero deltas are also retained. The pooled correlation's original Fisher-z interval is positive, so the *frozen* decision ladder gives rung 2: what the original decision ladder called "partial transfer," with an inconclusive primary mean. This historical label is not a present inference of positive test-selection optimism. That historical interval treats the 100 nested pairs as independent and is not cluster-robust. The new seed-cluster bootstrap is reported separately, not relabeled as preregistered: it resamples all 20 whole seed rows 100,000 times, retaining the five placements and recomputing each correlation (seed 20260908). Both primary intervals include zero; neither establishes a positive mean held-out gain at this protocol.

The archived derived value $\rho c_5 \sigma_{\text{Test}} = 0.1335$ ($\sigma_{\text{Test}} = 0.2518$) is retained only as a historical heuristic. Positive Pearson correlation does not establish the conditional regression and candidate-distribution assumptions needed for such a prediction. Selection among a shared baseline plus five placements is not selection among

Quantity	Estimate	95% CI
Original primary, seed-level t	$+0.1245$	$[-0.0406, 0.2896]$
Reviewer primary, seed-cluster percentile	$+0.1245$	$[-0.0180, 0.2830]$
Original pooled ρ, Fisher-z	0.4559	$[0.2850, 0.5987]$
Reviewer pooled ρ, seed-cluster percentile	0.4559	$[0.2268, 0.6372]$
Reviewer within-seed centred ρ, seed-cluster percentile	0.4386	$[0.2227, 0.6152]$
Historical heuristic $\rho c_5 \sigma_{\text{Test}}$	$+0.1335$	—
Honest mean test effect	-0.017	—
Non-baseline champion selection	17/20	—

Table 12: E1 readouts, with accuracy differences in percentage points. Original and reviewer-requested intervals use the explicitly named methods; only the latter are whole-seed percentile bootstraps. Within-seed centring removes shared additive baseline terms but changes variance normalization, so differences between the two correlations are not baseline-contribution fractions. The original rounded t critical value 2.093 reproduces the archived primary interval; the exact t_{19} quantile changes its endpoints by less than 0.000002 points.

five exchangeable candidates; merely substituting c_6 does not resolve that mismatch. The directly measured selected-test mean remains the primary bridge estimand.

Covariance and interval provenance. Under the uniform empirical distribution on 20 seeds and five placements, with all covariance terms normalized by population counts (ddof 0), the law of total covariance gives

$$\begin{aligned}\mathrm{Cov}_{s,j}(\Delta V, \Delta T) &= \mathbb{E}_s \,\mathrm{Cov}_j(\Delta V, \Delta T \mid s) + \mathrm{Cov}_s(\overline{\Delta V}_s, \overline{\Delta T}_s) \\ &= 0.0348025 + 0.0278674 = 0.0626699.\end{aligned}$$

The between-seed term contains seed-level effects and shared-baseline effects; it is not an identified causal baseline contribution. Thus the change from pooled 0.4559 to centred 0.4386 supports no "under 4%" baseline attribution. The previously printed main-text interval $[-0.019, +0.279]$ was labeled seed-clustered, but its exact seed/draw provenance was not recoverable from the archived analysis scripts. It is not the frozen t interval. We retain that frozen interval and separately report the new reproducible percentile interval, rather than selecting whichever interval appears more favorable. Complete seed readouts, input hashes, and calculation details are in `revision_results/statistics.json` and `docs/REVISION_STATISTICS.md`; the historical comparator and preregistration remain unchanged.

D.4 Where divergence starts: the traced MW-Net witness

A dense causal trace on the Meta-Weight-Net rig localizes the onset of divergence exactly (five frozen seeds; arms `base`, `shared_eval`, `isolated_eval`; all six preregistered gates pass: capture and trace conservation, isolated restoration, single probe firing, repeat floor, and two-draw replay), with one shuffled monitoring evaluation firing at step 1400 of 5400. Figure 16 reads the resulting layer-by-checkpoint relative-L2 grid cell by cell; what it establishes is that divergence begins at a single identified draw (the meta-batch index draw immediately after the probe, in all five seeds) and appears across the whole network at once rather than creeping in through a slower channel, and that restoring the stream cursors removes it bitwise.

D.5 Independent nanoGPT prediction and supplemental diagnostics

The model is a tinyShakespeare nanoGPT with six layers, six heads and embedding width 384, batch size 64, block length 256 and dropout 0.2. The endpoint score averages eight fixed validation batches drawn with seed 20260908. Execution uses deterministic FP32 with TF32 disabled, math SDPA, non-fused AdamW and no compilation. State checks cover weights/buffers, optimizer, scaler, prefetched batches, update count, learning rate, module modes, gradients and Python/NumPy/CPU/CUDA RNG at sparse checkpoints and the endpoint, not every update. The new campaign uses 16 calibration seeds (860901–860916) and 16 held-out seeds (860917–860932), five fixed placements and 2,000 updates. There are 704 main runs: 32 seeds with 11 paths each (baseline, five shared and five isolated), each run twice. Adding 22 runs for the separate pilot seed

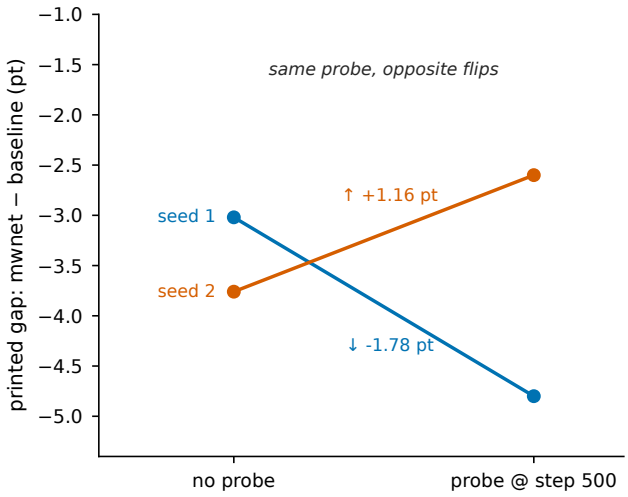


Figure 10: **The probe moves a printed method-versus-baseline comparison in opposite directions on two seeds (Meta-Weight-Net official code).** With a probe at step 500, the printed gap moves from −3.02 to −4.80 points on seed 1 (shift −1.78) and from −3.76 to −2.60 on seed 2 (shift +1.16). Both movements are far above the family's accuracy-exact repeat floor (Table 11). These bidirectional displacements illustrate the mechanism; Proposition 1 does not predict a sign or a zero mean across seeds.

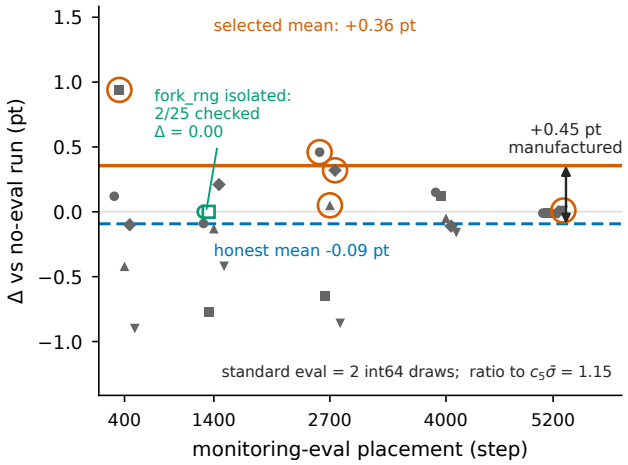


Figure 11: **The harvest reproduced by an *unmodified* standard monitoring evaluation, at a believable protocol (Meta-Weight-Net official code, full 45k train, 12 epochs).** A shuffled monitoring `DataLoader`'s two 64-bit seed draws desynchronize the pair; per-seed selection over five placements manufactures +0.45 points (tracking $c_5\bar{\sigma}$, ratio 1.15), while the same evaluation inside `fork_rng` is bitwise inert in the directly checked model/meta-network hash pairs (2 of 25 cells). This is scoped weight evidence, not a state-level check of all placements. The full 120-epoch campaign adds a same-codebase mechanism witness and three directly checked repaired pairs; its Gaussian agreement remains a small-sample, same-sample calibration.

860899 gives 726 raw records; the pilot is excluded from analysis. Reconstructing $D_{sj} = L_{s,\text{baseline}} - L_{sj,\text{shared}}$ from the raw fixed-panel losses exactly reproduces both 16×5 analysis grids. Neither repeat runs nor placement cells are treated as independent seeds.

The frozen primary prediction is $c_5\bar{s}/c_4(5) = 0.00388406$, with $c_5 = 1.16296447$, $c_4(5) = 0.93998560$ and calibration mean sample SD $\bar{s} = 0.00313936$. Its observed-minus-predicted discrepancy is -0.00045346 (approximate 95% interval $[-0.00134285, 0.00043370]$). The *prespecified uncorrected sensitivity*, $c_5\bar{s} = 0.00365096$,

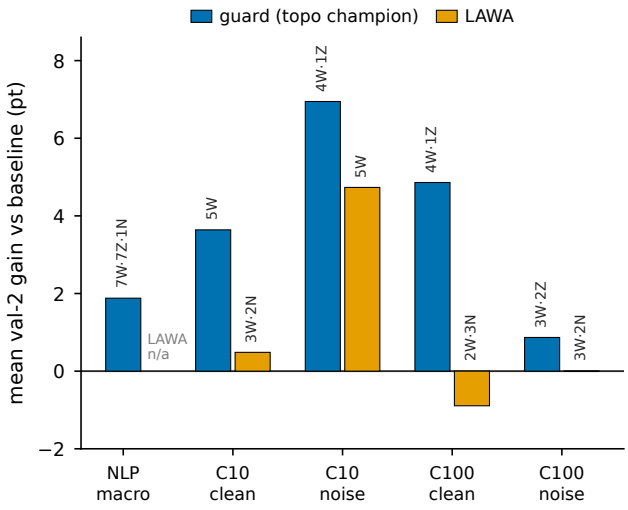


Figure 12: **Guard payout versus LAWA across the five under-tuned cells, as mean holdout gain over the paired baseline:** guard +1.88 (NLP macro), +3.64/+6.95 (CIFAR-10 clean/noise), +4.86/+0.87 (CIFAR-100 clean/noise); LAWA +0.48, +4.73, −0.89, −0.00 (not run on NLP). Above each bar: per-unit win/zero/negative counts (15 task–seed cells for NLP, 5 seeds per vision cell), where a zero is an *exact* 0.00 delta, not by itself a state-level certificate. The guard has one realized loss (−3.12, RTE seed 1); LAWA is negative in 2–3 units in three of the four vision cells. These observed payouts do not bound the guard's downside; the conditional no-op implication applies only under its premises.

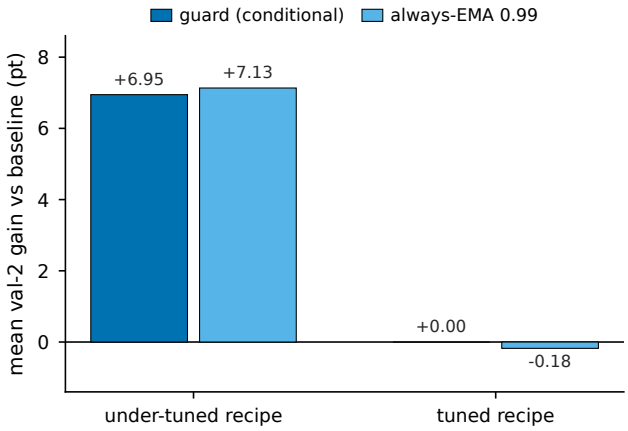


Figure 13: **The conditional guard against an unconditional always-EMA policy (CIFAR-10, 30% noise, deterministic, 5 seeds), as mean holdout gain over the paired baseline.** Under-tuned recipe: guard +6.95 versus always-EMA +7.13; tuned recipe: guard exactly 0.00 (all champions raw, metric-level zeros) versus always-EMA −0.18. The 0.18 points of upside conceded where EMA helps accompanies an observed zero where it hurts; these metrics alone do not establish a state-level certificate.

has prediction interval $[0.00307846, 0.00422724]$; relative to observed mean harvest 0.00343060, the discrepancy is -0.00022036 (-6.04%), with interval $[-0.00108275, 0.00064263]$. This sensitivity does not replace the primary. Both use 20,000 approximate percentile bootstrap draws (RNG seed 2026090801), independently resampling whole calibration and held-out seed vectors, generating calibration indices first. The intervals do not establish equivalence or identify a Gaussian residual law.

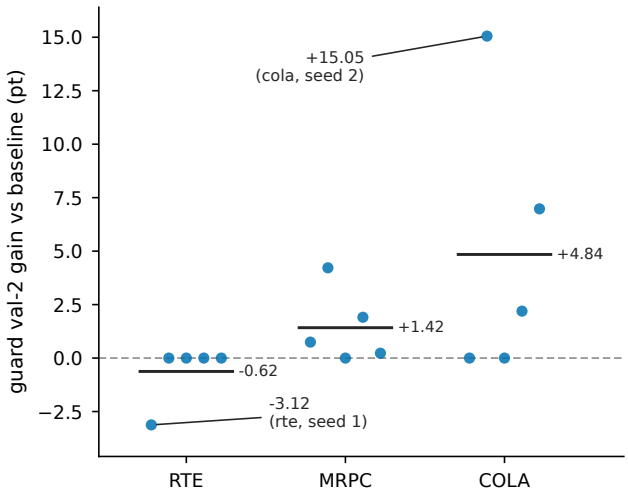


Figure 14: **Per-cell guard payout on the under-tuned NLP campaign (deterministic; 3 tasks × 5 seeds; champion from the selection log versus the paired baseline's final holdout metric).** Task means −0.62 (RTE), +1.42 (MRPC), +4.84 (CoLA); seven cells are exact zeros (champion raw or holdout tie), and the extremes are the single realized selection loss −3.12 (RTE seed 1) and the largest rescue +15.05 (CoLA seed 2). The −3.12 cell is the observed loss reported rather than averaged away.

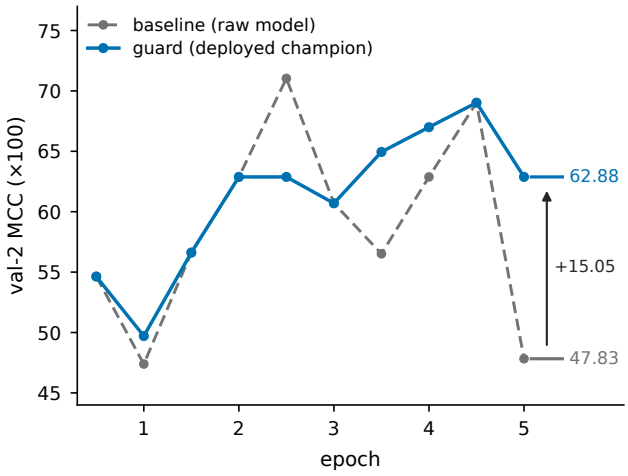


Figure 15: **The CoLA seed-2 rescue behind the campaign's +15.05 cell (deterministic pairing, same seed).** The baseline's raw model collapses on the final half-epoch (holdout MCC ×100: 47.83), while the guard's deployed champion (EMA 0.999, engaged by the epoch-1 fallback) ends at 62.88. The guard converts trajectory instability the baseline merely suffers into a selection among stable candidates, which is what the CoLA column of Figure 14 is made of.

Exploratory reported-score decomposition. The following diagnostics were added after read-out, unlike the sensitivity above. Calibration selected maximum, unselected placement mean and harvest are respectively 0.00462571, 0.00034492 and 0.00428079, with approximate 95% intervals $[0.00254966, 0.00678532]$, $[-0.00133125, 0.00201870]$ and $[0.00354967, 0.00502692]$. On held-out seeds they are 0.00324976, −0.00018085 and 0.00343060, with intervals $[0.00149894, 0.00490740]$, $[-0.00165377, 0.00128459]$ and $[0.00279334, 0.00407564]$. The selected fixed-panel mean's exploratory interval lies above zero. Selection and scoring use the same panel, so this is not an independent test improvement, and H is not identified as pure reporting optimism. The separate E1 validation-to-test bridge remains inconclusive.

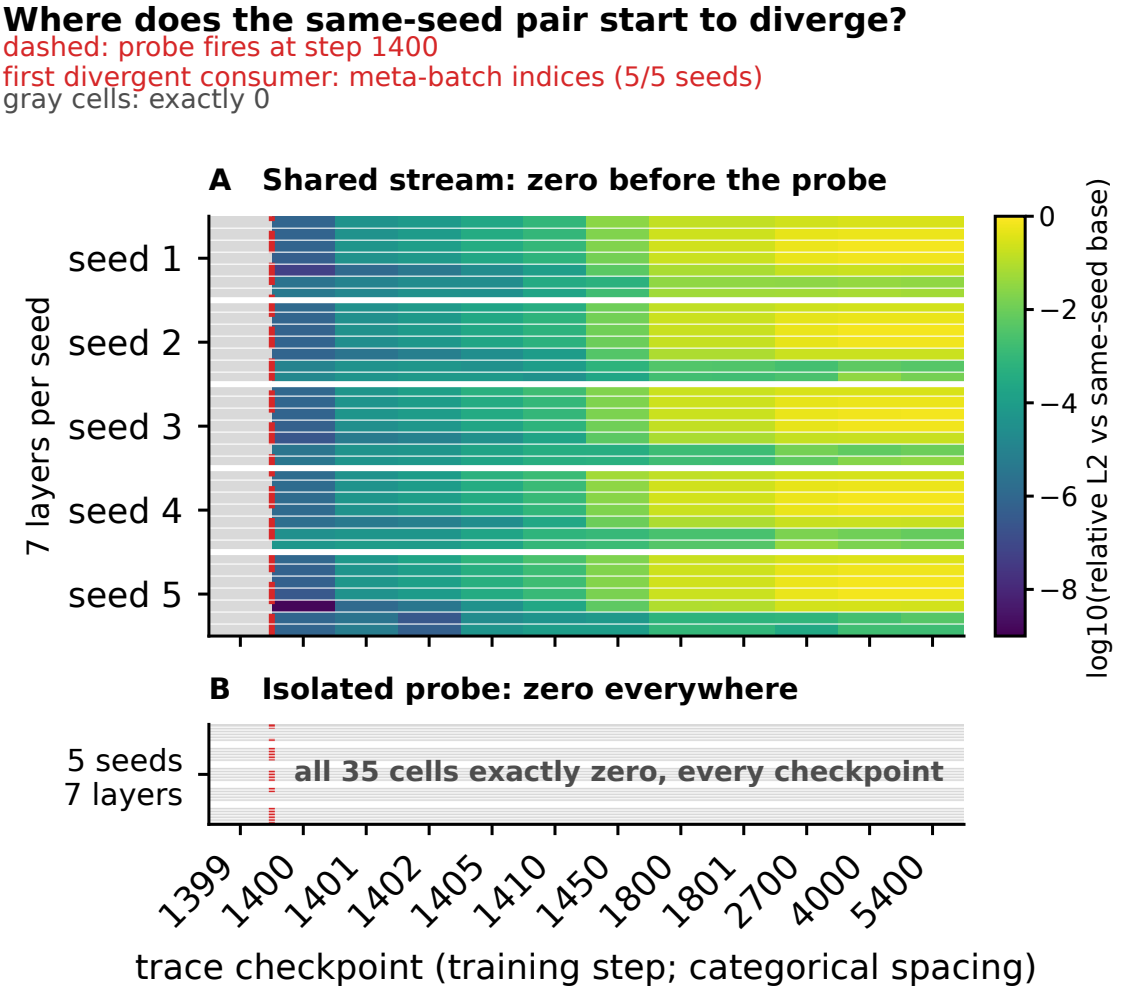


Figure 16: **Where the same-seed pair starts to diverge (MW-Net causal trace, five seeds).** Top: in the shared-stream arm, relative L2 against the same-seed base is exactly zero (gray) for every layer and seed at the pre-probe checkpoint and nonzero from the probe step (1400) onward; the first divergent training-visible consumer is the meta-batch index draw in all five seeds. Bottom: the isolated arm (the probe wrapped in a fork with full RNG-state restoration) is exactly zero everywhere through step 5400. Within each seed block, rows are the frozen layer order (stem, stages 1–3, head, then the meta-network's two layers); checkpoints are unevenly spaced and drawn categorically. Layer-wise magnitude localizes propagation without assigning causal responsibility for any final metric.

Exploratory placement and influence diagnostics. In placement order $200, 500, 900, 1300, 1700$, calibration column means (in 10^{-3} loss units) are $(2.644, 0.755, -1.054, -0.391, -0.230)$ and held-out means are $(-0.059, -0.371, 0.350, -0.355, -0.469)$. Corresponding sample variances (in 10^{-5} squared loss units) are $(4.274, 2.526, 1.028, 1.092, 1.125)$ and $(1.980, 2.383, 1.588, 1.255, 1.318)$. Raw off-diagonal covariance ranges are $[5.241, 23.483] \times 10^{-6}$ and $[3.304, 11.264] \times 10^{-6}$; correlations range $[0.382, 0.881]$ and $[0.210, 0.876]$. These raw dependencies can include a common seed component and do not by themselves reject conditionally iid residuals. Subtracting each seed's placement mean gives contrast covariance ranges $[-6.985, 4.606] \times 10^{-6}$ and $[-5.031, 3.462] \times 10^{-6}$, and correlation ranges $[-0.694, 0.791]$ and $[-0.690, 0.684]$. Centering mechanically imposes a sum-zero constraint; negative contrast correlations are not an iid rejection either. These small-sample descriptive diagnostics neither certify equal placement means/variances nor constitute a formal Gaussian test; full matrices are released in `revision_results/nanogpt_final/supplementary.json`.

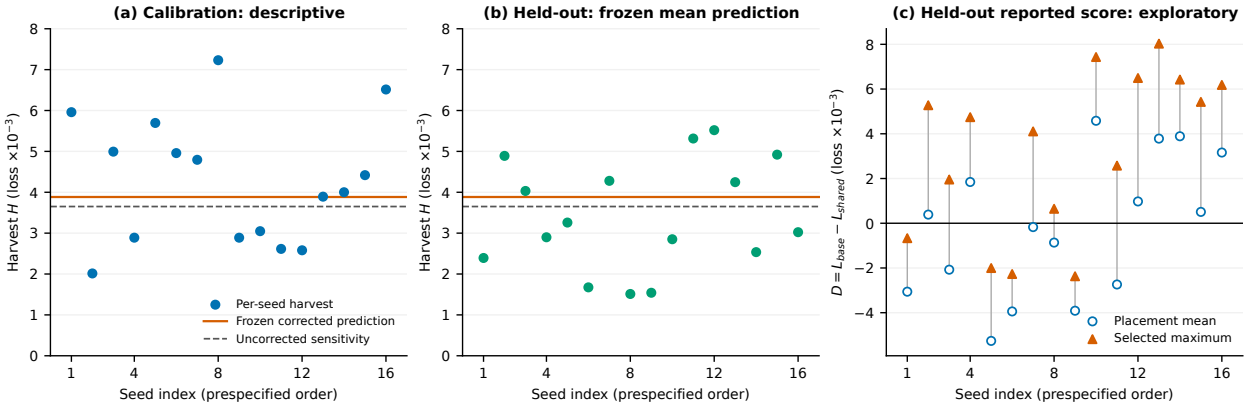


Figure 17: **Independent nanoGPT campaign, in prespecified seed order.** Left: descriptive calibration harvest. Middle: held-out harvest with the frozen corrected prediction and prespecified uncorrected sensitivity; horizontal lines predict a mean, not individual seeds. Right: exploratory held-out selected maximum and placement mean on the same fixed validation panel; connecting segments equal harvest. Repeats are not extra seeds. The figure does not demonstrate independent-test transfer.

Leaving out each calibration seed in turn gives corrected predictions $[0.00372860, 0.00400198]$. Leaving out each held-out seed, keeping the full frozen prediction fixed, gives observed means $[0.00329137, 0.00355852]$ and discrepancies $[-0.00059269, -0.00032554]$. These are influence ranges, not uncertainty intervals or reasons to exclude seeds; all seeds remain in the primary.

D.6 Reproducibility and data availability

The public repository `https://github.com/dumingyu2004/silent-unpairing-tmlr2026` is the artifact verified during review, with every later change listed in its `RELEASE_CHANGES.md`. It contains the raw results and analysis code used for the reported harvest, E1 and verification summaries, together with available run configurations, seeds, repeat records, exact upstream code versions and source citations. It includes all 726 individual records from the new nanoGPT campaign, its experiment and analysis scripts, frozen prediction, and per-seed checks. Reanalysis from a fresh clone reproduces the reported numerical results. Its README and file manifest state what is included; this is an analysis-reproduction package, not a claim that every historical training run can be repeated from it.

The repository also includes the two recovered historical MW-Net moneyshot runners, source-backed explanatory repair diffs, and fifteen original MW-Net endpoint files: baseline, repeat and isolated-probe model/meta-network states for five seeds. Their stored tensor bytes agree within each seed; these files do not contain complete optimizer, RNG or cursor state. The diffs are revision-generated explanations against documented source bases, not a recovered complete original repair-patch series. A separate local trace companion retains seventeen original MW-Net tensor archives for the five baseline/shared/isolated triplets and two baseline repeats, checked against their retained run records. These seventeen NPZ archives are not included in the public repository.

For E1, twenty split/noise input files are supplied as explicitly labelled reconstructions from the retained source, seeds and configuration records. The original runner generated these arrays in memory and exported only configuration and results; it did not separately save them. The reconstruction is checked against the source recipe, not against an unavailable original array file. Clean dataset labels are external inputs to the supplied corruption recipe. The repository therefore supports reanalysis and inspection of the specified inputs and repairs without claiming every original checkpoint or complete historical retraining coverage.

Historical writing and editing records reproduce the retained E1 runner and protocol text exactly after newline normalization. The last content-changing protocol edit is recorded at 10 July 2026, 14:56:39 UTC (11 July locally), about 51 hours before the recorded campaign launch at 12 July 2026, 17:39:29 UTC.

The repository includes a redacted chronology and verification record in `docs/E1_PROVENANCE.md` and `revision_results/e1_provenance.json`. For the new nanoGPT campaign, local controller records place the prediction freeze at 8 September 2026, 18:55:00 UTC, before launching the held-out runs. The available protocol and timing records are included with their provenance. These are author-maintained pre-launch records, not independent external registration; the chronology is based on recorded events rather than copied-file dates.

D.7 Extended monitoring and full-protocol harvest witnesses

The harvest also does not need an inserted probe. Replacing the semantically inert permutation draw with an *unmodified* standard monitoring evaluation (switch the model to evaluation mode, iterate a shuffled validation loader for a few batches, switch back) reproduces the mechanism at a believable protocol (Meta-Weight-Net official code, the full 45,000-image training set, 12 epochs on the real three-phase schedule, held-out test). The shuffled loader draws exactly *two* 64-bit integers from the global stream training consumes, its loader's base seed and its sampler's seed, with nothing inserted, and that alone desynchronizes the pair: across 5 seeds $\times 5$ placements, per-seed selection manufactures $+0.45$ points over the honest mean at ratio 1.15 to $c_5\bar{\sigma}$. The two directly checked isolated comparisons (two seeds, placement 1400) match the never-evaluated baseline in final model and meta-network parameters and buffers, with test-accuracy deltas exactly 0.00 (Figure 11). The remaining 23 repaired comparisons were not directly verified; these two endpoints do not establish zero selection gain across the placement grid. Together with the state-purity inspection and controlled intervention, these repaired comparisons support the stream-consumption explanation. Endpoint identity alone cannot rule out transient changes that later reconverge. The effect is small at this protocol and its size is not the claim: two integers drawn by a routine validation loop suffice to manufacture a selectable, extreme-value-sized gain, and the isolation wrapper restores endpoint equality in the two checked pairs.

The full official protocol closes the loop. At 120 epochs (58,800 steps; repeat floor SHA-exact across all three seeds, model and meta-network alike), the same rig sustains every layer: all twelve placement$\times$seed pairs desynchronize bitwise, a single inert permutation draw moves the printed method-versus-baseline gap by up to 0.32 points on the paper's best-accuracy metric (0.56 on final), and per-seed best-of-four selection manufactures $+0.16$ points over the honest mean, with $c_4\bar{\sigma} = 0.16$ at ratio 1.0075 using exact quadrature. These are only three seeds; best and final are endpoints of the same runs, not independent replications. The full plug-in simulations in Appendix C do not discriminate Gaussian from t_3 residuals. The same draw inside a forked RNG state restores bitwise identity with the never-probed baseline wherever we checked it (three of the twelve arms, verified by model/meta-network SHA-256). Those three checked repaired deltas are zero; we do not infer an all-placement bitwise check. Magnitudes anneal with the schedule, as in the BHN witness; the measured mechanism and scoped repair persist, while the Gaussian calibration remains conditional.

E Environment Manifest

- Python 3.12; PyTorch `2.11.0+cu128`; `transformers` 5.12.1; `accelerate` 1.14.0; NumPy 2.4.4.
- Hardware: unless stated otherwise, a single NVIDIA GeForce RTX 5080 produced the registered floors and paired numbers. E1 used a separate Linux host with one NVIDIA RTX PRO 6000 Blackwell, driver 595.71, and the same PyTorch `2.11.0+cu128` build; its repeat-floor gate was rerun there before launch and passed at model and meta-network SHA-256 resolution.
- The new nanoGPT calibration/held-out campaign used an NVIDIA RTX PRO 6000 Blackwell Workstation Edition with PyTorch `2.11.0+cu128`; its per-path repeat checks were performed on that stack, not inferred from the default RTX 5080 rig.
- Framework citations in Appendix B are pinned to `transformers` 5.12.1 and `torch` 2.11.0; audited repositories are pinned by commit SHA (Tables 2 and 3).
- `scripts/rng_premise_check.py`: 13/13 assertions pass on this stack, exercising both the CPU and the CUDA device.

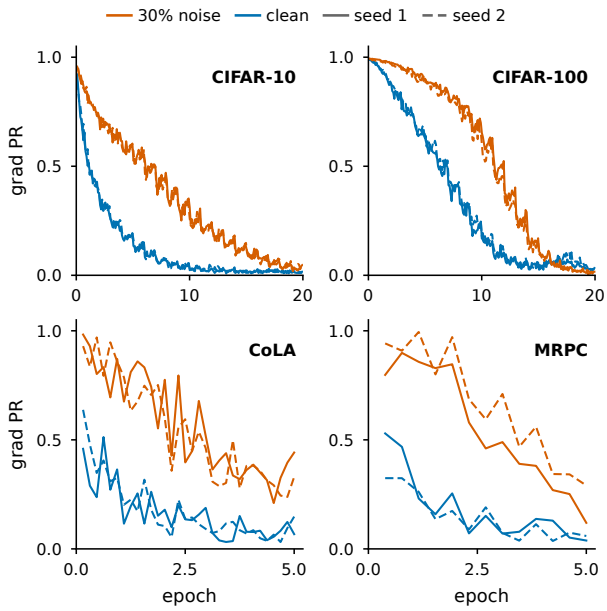


Figure 18: **The participation-ratio detector transfers across datasets and modalities:** same-seed clean (blue) versus 30%-noise (vermillion) runs, two seeds each (solid/dashed), on CIFAR-10/100 (per-step monitor) and CoLA/MRPC (per-40-step monitor). All 8/8 clean–noise pairs separate in the early window; at the sample nearest epoch 1 the clean-versus-noise gaps are +0.27 (CIFAR-10), +0.52 (CoLA), and +0.68 (MRPC), while CIFAR-100's separation (+0.04 at epoch 1) opens over the following epochs. Readings are same-dataset and seed-paired only: the scales differ across datasets, the no-global-cutoff boundary.

- **Scope of bitwise claims.** A bitwise floor checks repeatability of its recorded state on its tested execution path, not pairing with a different arm. Hardware, driver, or library changes can change the bits; repeat checks must therefore be rerun per materially distinct path and stack. The protocol is portable, not a guarantee of cross-platform bit identity.

F Supplementary Monitor Figures

G Extended context and discussion

G.1 Connections to prior work

Reproducibility, nondeterminism, and causally valid comparison. Empirical work documents how small randomness sources amplify into large run-level differences: a one-bit weight change drives a different final model (Summers & Dinneen, 2021), earlier perturbations diverge more (mirroring our finding that a probe's *timing* governs its effect) (Altıntaş et al., 2025), exact reproducibility is geometrically fragile (Ly & Gong, 2025), and seeds and data order swing results in reinforcement and active learning (Henderson et al., 2018; Ji et al., 2023; Patterson et al., 2024). A parallel line treats this randomness as *variance* to be budgeted across initializations, data orders, and hyperparameters, then averaged over with more seeds (Dodge et al., 2020; Bouthillier et al., 2021). That line treats performance as a distribution over seeds rather than a point estimate, and we complement it from the other side: nothing here argues against reporting distributions, but same-seed pairing is how two methods get compared without paying for many seeds, and we show it can be invalidated from inside the method. Desynchronization alone need not change the marginal mean effect; its variance consequence depends on the coupling covariance. Selecting favorable outcomes on the reported quantity adds directional optimism (Proposition 2), which averaging already-selected outcomes across seeds need not remove. The +4.01 demonstration uses this rule. Unlike seed hacking, the divergence is manufactured

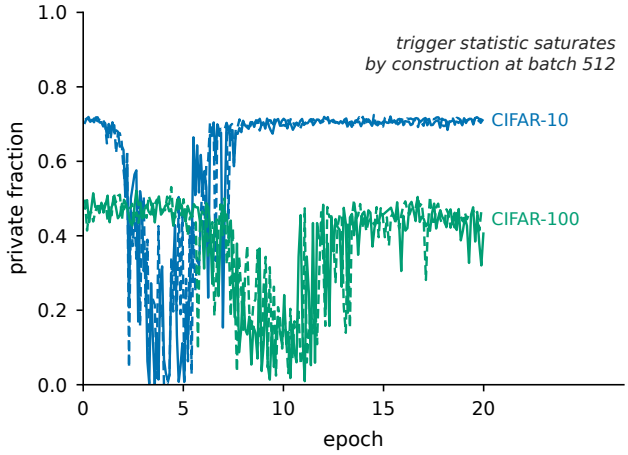


Figure 19: **The original private-fraction trigger statistic at batch 512 (deterministic clean campaigns, seeds 1–2):** the statistic sits pinned at its saturation plateau, run medians 0.70 (CIFAR-10) and 0.44 (CIFAR-100), apart from a transient mid-training dip window, and carries no memorization signal at this batch size. This is the saturation diagnosis (levels ≈0.68 and ≈0.45): the natural trigger never fires in the vision suite, and the epoch-1 fallback engages the candidates instead.

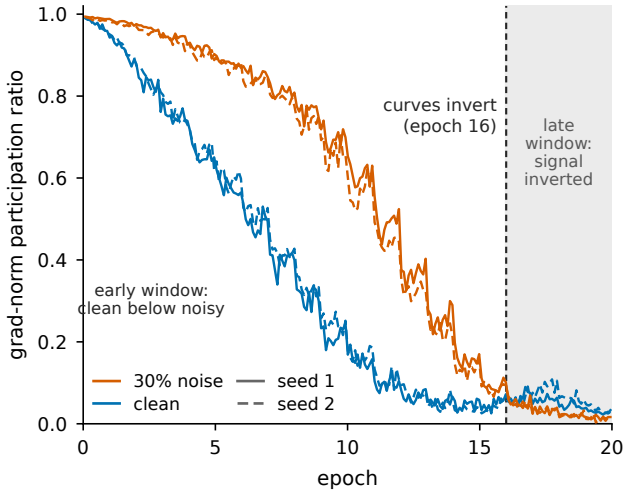


Figure 20: **The detector's operating window is early training (CIFAR-100 monitor study, full runs).** The clean and 30%-noise conditions are separated from the first epoch (two-seed condition means 0.918 vs. 0.978), but as the training loss collapses the noisy curves fall *below* the clean ones (the condition means invert at epoch 16), so late-window readings carry the signal with the wrong sign: readings past the early epochs carry no information, the late-window boundary.

inside the method by its own probe and harvested by its own selection step, with no researcher-side seed choice anywhere. It inherits the statistics of model-selection over-fitting (Cawley & Talbot, 2010) but not its source: isolation removes the probe-induced displacement at the checked resolution under the pairing premises.

The closest prior work sits in stochastic simulation, where misaligned random-number realizations are a named problem: Stout & Goldie (2008) prescribe synchronizing the numbers consumed by identical stochastic events; Klein et al. (2024) quantify the damage when that alignment fails, reporting order-of-magnitude variance reductions once comparisons draw aligned numbers; and Buffalo et al. (2026) formalize the causal failure (under a stateful PRNG a path-altering intervention can shift draw indices on the aligned prefix,

voiding common-random-number counterfactuals) and repair it with counter-based RNG. We build on this common-random-number alignment principle; our contribution is its diagnosis in deep-learning evaluation, released-code causal witnesses, and a connection to selection on the displaced quantity. Nor is isolation tooling the gap: splittable keys are a JAX design principle (JAX PRNG), PyTorch ships a fork-rng context manager (PyTorch fork rng), the mechanism surfaces in issue trackers (PyTorch Lightning contributors, 2022) and RL tooling (The MathWorks, Inc., 2024), and the common-random-number lineage is decades old (Glasserman & Yao, 1992; Pearce et al., 2022). We do not claim a new general principle of randomness or model selection. Section 2 connects these established ideas through a checkable deep-learning mechanism and separates reporting optimism from an unestablished held-out test improvement.

Reproducibility tooling and the harmlessness view. A systems literature makes bitwise-reproducible training practical and supplies the operational half of our repair: record–replay and deterministic-execution frameworks (Chen et al., 2022; Heumos et al., 2023) and verified bit-identical pipelines (Shkolnikov, 2026) show that an exact repeat floor is attainable in practical systems. We use these tools rather than compete with them: the contribution is not a determinism mechanism but the observation that *pairing validity*, not reproducibility of a single run, is the property at stake. Closest in spirit, Zhuang et al. (2022) characterize how tooling injects nondeterminism; we localize a break in the paired comparison and demonstrate its directional consequence when selection acts on the affected outcome. Jordan (2024) argues that the between-run test-set variance it measures is harmless and inevitable, which we do not dispute: pairing exists precisely to see *through* that variance, and our result is that the instrument can fail, while selection on the reported quantity creates optimism; observed uplift can also contain systematic placement effects.

Bilevel meta-learning and sample reweighting. Learning-to-reweight methods learn per-example weights through a virtual optimizer step and a meta-objective on held-out data: L2RW (Ren et al., 2018), Meta-Weight-Net (Shu et al., 2019), the co-training lineage of Co-teaching (Han et al., 2018) and DivideMix (Li et al., 2020), recent entries BHN (Yu et al., 2023) and RENT (Bae et al., 2024), and a three-phase reweighting theory (Zhang et al., 2025). This is the audit's home neighborhood. A second, appendix-scoped diagnosis speaks to this family: the canonical lr-scaled virtual step emits an exactly η-scaled meta-signal, which can be dominated where an η-independent term of comparable or larger size competes for the same weights, a magnitude ratio rather than an inevitability; Appendix C.5 gives the statement, the three escapes, and a per-system taxonomy, with the proof in Appendix C.

Weight averaging. Our guarded averaging endpoint illustrates the conditional no-op, rather than claiming an accuracy advantage over SWA, model soups, trainable/early averaging, LAWA, or adaptive averaging (Izmailov et al., 2018; Wortsman et al., 2022; Li et al., 2023; Sanyal et al., 2024; Kaddour, 2022; Demir et al., 2024; Melis, 2022; Wang et al., 2025; Ajroldi et al., 2025). The auxiliary comparisons are retained in the appendix.

G.2 Practical implications and detailed scope

A norm that would close the gap. Probe consumption of a stream later used by training can break trajectory alignment. Proposition 1 gives a sufficient preservation condition and localizes update-relevant divergence; mere stream sharing is not enough. The Trainer's sequential-sampler/no-grad evaluation is stream-disjoint from training in the tested configuration (Table 6), and frameworks that isolate probe RNG (Lightning's isolate-rng utility (Falcon & The PyTorch Lightning team, 2019), fairseq's re-keying (Ott et al., 2019), JAX's splittable keys (JAX PRNG)) ship the primitives that enforce isolation, though whether a given program is isolated still depends on how it assigns probe and training keys. Isolation is often small as a code change, but it must be *checked*, not assumed: deterministic kernels and a per-path repeat/state audit add real runtime and storage costs, and some platforms lack deterministic implementations. The three practices of Section 2.5 make the check mechanical. A venue-level norm of repeat-floor reporting would make this artifact class checkable, not cost-free.

Selection optimism. The general selection decomposition separates reporting optimism from systematic placement effects without independence assumptions. Selecting the best of m candidates has expected uplift

$c_m\sigma$ only under the stated iid-Gaussian benchmark (Proposition 2); correlated, heteroscedastic, or offset placements need not obey that scale: classical selection bias (Cawley & Talbot, 2010), fed here by divergence the instrument manufactures. Observed max-minus-mean uplift is not identified as pure optimism or held-out gain: it also contains placement effects. The no-op corollary is an application of the pairing condition, not a claim that selecting or averaging candidates improves accuracy. Its auxiliary candidate-selection analyses remain separate from the core evaluation diagnosis.

Distinct safeguards and remaining limits. State-pure isolation preserves coupling under the proposition's premises. Independent-seed comparisons have no exact coupling to lose, but can still suffer selection optimism. Omitting selection removes this source of directional optimism, not the displacement or its possible variance effect. A large true effect does not restore pairing. Every measurement here is single-process, and we have not tested the artifact under distributed training of any topology, which we leave as future work. Our method claims are at academic scale (ResNet-18, RoBERTa-base; nanoGPT enters only the artifact analysis) and cover label noise rather than distribution drift, and in tuned regimes the reported guard gain is metric-zero; historical evidence does not establish a state-level certificate for every cell. The artifact's *dynamic* evidence spans CIFAR-10 convnets, two GLUE tasks on RoBERTa-base, and nanoGPT at 200 iterations and a separate 2,000-update calibration/held-out campaign, with nothing claimed at ImageNet or production-LLM scale. The latter freezes its scale on 16 calibration seeds and checks a mean on 16 held-out seeds; it does not validate a universal Gaussian model, per-run prediction, or independent-test transfer (Section 2.4). The core eleven-of-eleven audit is double-read, and seven of its entries and the entire field-wide wave are static findings, labeled as such, with pinned commits released for re-execution.